



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Tunis El Manar
Faculté des Sciences de Tunis
Département des Sciences de l'Informatique



RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du
Diplôme de Licence en Ingénierie des Systèmes Informatiques "Computer
Engineering" Spécialité : Systèmes Embarqués et Internet des Objets

Par
Mohamed Amine Ghannouchi

Sécurisation des flux de communication pour un écosystème IoT émulé

Organisme d'accueil : Tunisie Telecom

Soutenu le : 08/06/2026, devant le jury :

Président : Mr. Taha BENNANI

Rapporteur : Mm. Saloua ZAMMELI

Encadrant Académique : Mm. Nourhene ELLOUZE

Encadrant Professionnel : Mr. Hatem JEMAI et Mr. Moez KHLIF

IOT - 3126017

Année Universitaire : 2025–2026

Dédicace

*À mes parents, pour leur soutien indéfectible
et leur confiance tout au long de ce parcours.*

*À mes enseignants de la Faculté des Sciences de Tunis
qui m'ont transmis la passion de l'informatique.*

*À tous ceux qui œuvrent pour la sécurité
des systèmes d'information de demain.*

Remerciements

Ce travail n'aurait pu aboutir sans le concours de nombreuses personnes à qui je tiens à exprimer ma gratitude.

Je remercie tout d'abord **Mme. ELLOUZE NOURHENE**, encadrante universitaire à la Faculté des Sciences de Tunis, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de ce projet.

Mes remerciements vont également à **Mr. Hatem JEMAI** et **Mr. Moez KHLIF** de *Tunisie Telecom* pour m'avoir accueilli au sein de leur équipe, orienté vers des problématiques réelles de l'industrie télécom et soutenu dans mes expérimentations techniques.

Je remercie l'ensemble du corps enseignant du **Département Informatique de la FST** pour la qualité de la formation dispensée.

Un grand merci aux équipes des projets open-source **Wazuh**, **HashiCorp Vault**, **Mosquitto** et **Suricata** dont les documentations exhaustives ont grandement facilité l'implémentation.

Enfin, je remercie ma famille et mes amis pour leur encouragement constant durant cette année académique.

Table des matières

Dédicace	i
Remerciements	ii
Liste des acronymes	ix
Introduction générale	1
1 Contexte général et état de l'art	2
1.1 Introduction	2
1.2 Présentation de l'organisme d'accueil	2
1.2.1 Service d'accueil	3
1.2.2 Positionnement de Tunisie Telecom face aux enjeux IoT	4
1.3 Cadre général du projet	4
1.4 L'IoT dans les réseaux de télécommunications	4
1.4.1 Croissance et enjeux du marché IoT	4
1.4.2 Rôle des opérateurs télécom dans l'écosystème IoT	5
1.5 Menaces et vulnérabilités spécifiques à l'IoT	5
1.5.1 Surface d'attaque étendue	5
1.5.2 OWASP IoT Top 10 et menaces réseau	6
1.5.3 Études de cas de référence	6
1.6 Technologies et protocoles de sécurisation	6
1.6.1 Protocoles applicatifs IoT	7
1.6.2 Sécurisation du transport : TLS 1.3 et mTLS	7
1.6.3 Gestion des identités : PKI et automatisation	8
1.6.4 Détection d'intrusion et supervision	8
1.6.5 Apprentissage automatique pour la détection d'anomalies	10
1.7 Étude de l'existant et analyse critique	10
1.7.1 État des lieux de la sécurité IoT dans les déploiements opérateur	11
1.7.2 Analyse critique des solutions existantes	11
1.7.3 Synthèse du diagnostic	11
1.8 Problématique	12
1.9 Objectifs du projet	12
1.9.1 Objectif général	13
1.9.2 Objectifs spécifiques	13
1.10 Besoins fonctionnels	13
1.11 Périmètre et contraintes du projet	13

1.11.1	Périmètre fonctionnel	13
1.11.2	Contraintes du projet	14
1.11.3	Hypothèses de travail	14
1.12	Démarche méthodologique	14
1.13	Conclusion	15
2	Environnement technique du laboratoire	16
2.1	Introduction	16
2.2	Vue d'ensemble du laboratoire	16
2.3	Infrastructure réseau	18
2.3.1	pfSense — pare-feu et passerelle WAN	18
2.3.2	MikroTik CHR — routeur L3 et segmentation VLAN	19
2.3.3	Politique de sécurité inter-zones	19
2.4	Composants applicatifs	20
2.4.1	Broker MQTT Mosquitto (zone DMZ)	20
2.4.2	HashiCorp Vault (zone PKI)	21
2.4.3	Simulateur de flotte IoT — fleet-sim (zone IoT)	21
2.4.4	Wazuh Manager (zone SIEM)	22
2.4.5	OpenSearch (zone SIEM)	23
2.4.6	OpenSearch Dashboards (zone SIEM)	23
2.4.7	Suricata IDS (zone Analyse)	23
2.4.8	Moteur ML — ml-engine (zone Analyse)	24
2.4.9	Boîte à outils d'attaque — attackbox (zone IoT)	24
2.5	Synthèse des interactions entre composants	25
2.6	Conclusion	25
3	Conception	26
3.1	Introduction	26
3.2	Spécification des besoins	26
3.2.1	Besoins fonctionnels (BF)	26
3.2.2	Besoins non fonctionnels (BNF)	26
3.2.3	Acteurs du système	26
3.2.4	Diagramme de cas d'utilisation	28
3.3	Vue d'ensemble de la solution proposée	30
3.4	Planification des sprints	33
3.5	Backlog produit	34
3.6	Modélisation UML	34
3.6.1	Diagramme de classes du simulateur	34
3.6.2	Diagramme de séquence : émission d'un certificat client	36
3.6.3	Diagramme d'activité : cycle de publication mTLS	36

3.6.4	Diagrammes d'activité des scénarios d'attaque	38
3.7	Conclusion	40
4	Réalisation	41
4.1	Introduction	41
4.2	Environnement de développement	41
4.3	Architecture globale et topologie réseau	41
4.3.1	Topologie réseau	41
4.4	Infrastructure à clés publiques (Vault)	44
4.4.1	Hiérarchie	44
4.4.2	Rôles Vault et émission	46
4.5	Broker MQTT en TLS 1.3 + mTLS	46
4.6	Scénarios d'attaque et contre-mesures	48
4.7	Pipeline d'apprentissage automatique	51
4.7.1	Données et features	53
4.8	Conclusion	54
5	Tests, résultats et validation	55
5.1	Introduction	55
5.2	Plan de tests	55
5.2.1	Tests unitaires et vérifications préalables	55
5.3	Résultats de sécurité	56
5.3.1	Preuves visuelles	56
5.4	Résultats du module ML	59
5.4.1	Isolation Forest	59
5.4.2	Random Forest	60
5.5	Résultats de performance	62
5.6	Limites identifiées	64
5.7	Perspectives	65
5.8	Conclusion	65
	Conclusion générale	66
	Références bibliographiques	68

Table des figures

1.1	Logo de Tunisie Telecom	2
1.2	Organigramme de Tunisie Telecom	3
2.1	Topologie du laboratoire GNS3	16
2.2	Topologie réelle dans GNS3	18
3.1	Diagramme de cas d'utilisation global	29
3.2	Vue d'ensemble de l'architecture	32
3.3	Architecture générale du système	33
3.4	Diagramme de Gantt	34
3.5	Diagramme de classes du simulateur	35
3.6	Séquence d'émission d'un certificat client	36
3.7	Cycle de publication mTLS	37
3.8	Scénario S5 : flood CONNECT	38
3.9	Scénario S6 : compromission de certificat	39
4.1	Topologie réseau	42
4.2	Flux E2E — Phase 2 : connexion mTLS	43
4.3	Flux E2E — Phase 4 : détection et supervision	44
4.4	Hiérarchie PKI à deux niveaux	45
4.5	Handshake TLS 1.3 mTLS	47
4.6	Chaîne de détection Suricata–Wazuh	49
4.7	Pipeline d'ingestion des alertes	50
4.8	Pipeline ML	52
4.9	Distribution des classes	53
4.10	Matrice de corrélation des features	54
5.1	Règles de filtrage pfSense	56
5.2	Campagne à 100 dispositifs	57
5.3	Scénario S5 : flood CONNECT	58
5.4	Alertes Suricata	58
5.5	Tableau de bord Wazuh	59
5.6	Matrice de confusion Isolation Forest	59
5.7	Distribution des scores Isolation Forest	60
5.8	Confusion et ROC Random Forest	60
5.9	Confusion multi-classes Random Forest	61
5.10	Importance des features	61
5.11	Distribution du handshake mTLS	62

5.12 Latence de connexion	63
5.13 Débit applicatif	63
5.14 RTT par taille de message	63
5.15 Comparaison des protocoles MQTT	64

Liste des tableaux

1.1	Comparaison des principaux protocoles applicatifs IoT	7
1.2	Comparaison des solutions IDS open source	9
1.3	Comparaison des solutions SIEM	10
1.4	Diagnostic des lacunes et briques de réponse correspondantes	12
1.5	Objectifs spécifiques du projet	13
1.6	Périmètre du projet – inclus et exclus	14
2.1	Inventaire des nœuds du laboratoire GNS3	17
2.2	Politique de filtrage inter-VLAN (<i>deny-by-default</i>)	20
3.1	Besoins fonctionnels	26
3.2	Besoins non fonctionnels	27
3.3	Correspondance lacunes – briques de sécurité – objectifs spécifiques . .	31
3.4	Planification des sprints	33
4.1	Scénarios d’attaque, détection et contre-mesures	48
5.1	Plan de tests	55
5.2	Taux de détection des scénarios d’attaque (10 rejeux par scénario) . . .	56
5.3	Performances ML (validation 5-fold)	62

Liste des acronymes

IoT	Internet of Things – Internet des Objets
TLS	Transport Layer Security
mTLS	Mutual TLS – TLS mutuel (authentification bidirectionnelle)
DTLS	Datagram TLS
PKI	Public Key Infrastructure – Infrastructure à Clés Publiques
CA	Certificate Authority – Autorité de Certification
CSR	Certificate Signing Request
CRL	Certificate Revocation List
OCSP	Online Certificate Status Protocol
MQTT	Message Queuing Telemetry Transport
CoAP	Constrained Application Protocol
HTTP	HyperText Transfer Protocol
AMQP	Advanced Message Queuing Protocol
SIEM	Security Information and Event Management
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
DPI	Deep Packet Inspection
DoS	Denial of Service
DDoS	Distributed Denial of Service
MiTM	Man-in-the-Middle
ML	Machine Learning – Apprentissage Automatique
RF	Random Forest
IF	Isolation Forest
ROC	Receiver Operating Characteristic

AUC	Area Under the Curve
RTT	Round-Trip Time
QoS	Quality of Service
DMZ	DeMilitarized Zone
VLAN	Virtual Local Area Network
GNS3	Graphical Network Simulator 3
VM	Virtual Machine
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ACL	Access Control List
CN	Common Name
SAN	Subject Alternative Name
E2E	End-to-End
RSA	Rivest–Shamir–Adleman
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
OVA	Open Virtual Appliance
TT	Tunisie Telecom
FST	Faculté des Sciences de Tunis
OWASP	Open Worldwide Application Security Project
ENISA	European Union Agency for Cybersecurity
NIST	National Institute of Standards and Technology
RFC	Request For Comments
ADSL	Asymmetric Digital Subscriber Line
LPWAN	Low-Power Wide-Area Network

SOC Security Operations Center

ETL Extract Transform Load

BF Besoin Fonctionnel

BNF Besoin Non Fonctionnel

O Objectif

Introduction générale

Les objets connectés occupent aujourd’hui une place importante dans la vie de tous les jours. On les retrouve dans les maisons, les hôpitaux, les usines, les transports ou encore dans les services publics. Leur rôle est simple : observer une situation, envoyer une information et aider à réagir plus vite. Ils permettent ainsi de gagner du temps, d’améliorer le suivi de certaines activités et de rendre plusieurs services plus pratiques.

Cependant, plus les appareils échangent d’informations, plus les risques augmentent. Si ces échanges sont mal protégés, une personne malveillante peut lire des messages, se faire passer pour un appareil autorisé, perturber un service ou récupérer des données qui ne devraient pas être partagées. La sécurité devient alors une condition essentielle pour préserver la confiance, assurer le bon fonctionnement du service et protéger les informations échangées.

Ce projet de fin d’études a pour but de proposer une manière claire et complète de mieux protéger ces échanges. L’idée est de ne pas compter sur une seule protection, mais d’en réunir plusieurs : vérifier qui communique, protéger les messages pendant leur envoi, limiter ce que chacun a le droit de faire et surveiller le système pour repérer rapidement ce qui semble anormal. Le travail a été réalisé dans un cadre de test proche d’une situation réelle, afin de vérifier à la fois l’efficacité de la protection proposée et sa facilité d’utilisation.

Ce rapport est organisé en cinq chapitres. Le premier réunit le contexte général et l’état de l’art : présentation de l’organisme d’accueil, enjeux du marché IoT, cartographie des menaces, protocoles et mécanismes de sécurisation, étude de l’existant, problématique, objectifs, besoins fonctionnels et démarche méthodologique. Le deuxième décrit l’environnement technique du laboratoire en présentant chaque composant de la topologie GNS3 : infrastructure réseau, broker MQTT, PKI, simulateur de flotte, chaîne de détection et boîte à outils d’attaque. Le troisième formalise la conception de la solution : spécification des besoins, acteurs du système, cas d’utilisation et architecture de défense en profondeur. Le quatrième décrit la réalisation de chaque brique technique et les scénarios d’attaque. Enfin, le cinquième regroupe les tests, les résultats, les limites et les perspectives d’évolution, avant la conclusion générale. Cette progression conduit volontairement d’une présentation accessible des enjeux vers une analyse technique et expérimentale du projet et de ses résultats.

Chapitre 1

Contexte général et état de l'art

1.1 Introduction

Ce chapitre pose les fondations du projet de fin d'études en réunissant le cadre institutionnel, le contexte technologique et le socle scientifique nécessaires à la compréhension de la solution proposée. Il présente d'abord l'organisme d'accueil et son positionnement stratégique face aux enjeux de la sécurité IoT, puis situe le travail dans le contexte du marché IoT et des opérateurs télécom. Le chapitre dresse ensuite un état de l'art des menaces spécifiques à l'IoT, recense les technologies et protocoles de sécurisation disponibles et analyse les travaux existants afin d'en identifier les limites. Sur cette base, il formalise la problématique, définit les objectifs, expose les besoins fonctionnels, délimite le périmètre de l'étude et explicite la démarche méthodologique retenue.

1.2 Présentation de l'organisme d'accueil

Tunisie Telecom est l'opérateur historique de télécommunications en Tunisie. Fondée en 1995, l'entreprise joue un rôle clé dans le développement des infrastructures de communication du pays et dans la fourniture de services de téléphonie, d'Internet et de transmission de données [1]. La figure 1.1 présente le logo de Tunisie Telecom.



Fig. 1.1 : Logo de Tunisie Telecom

L'opérateur propose une large gamme de services destinés aussi bien aux particuliers qu'aux entreprises, incluant :

- la téléphonie fixe et mobile ;
- l'accès à Internet haut débit et très haut débit ;
- les services de données et de connectivité ;
- les solutions cloud et les services numériques.

Grâce à un vaste réseau d'infrastructures couvrant l'ensemble du territoire national, Tunisie Telecom contribue activement à la transformation numérique du pays. L'entreprise investit également dans des technologies émergentes telles que la 5G, l'Internet des objets et les solutions de cybersécurité afin de répondre aux besoins croissants en connectivité et en sécurité.

1.2.1 Service d'accueil

La structure organisationnelle de Tunisie Telecom est constituée d'une direction générale, de neuf directions centrales et de 24 directions régionales. Chacune des parties prenantes de l'organisation joue un rôle essentiel dans le fonctionnement de l'ensemble. Cette organisation est illustrée dans la figure 1.2.

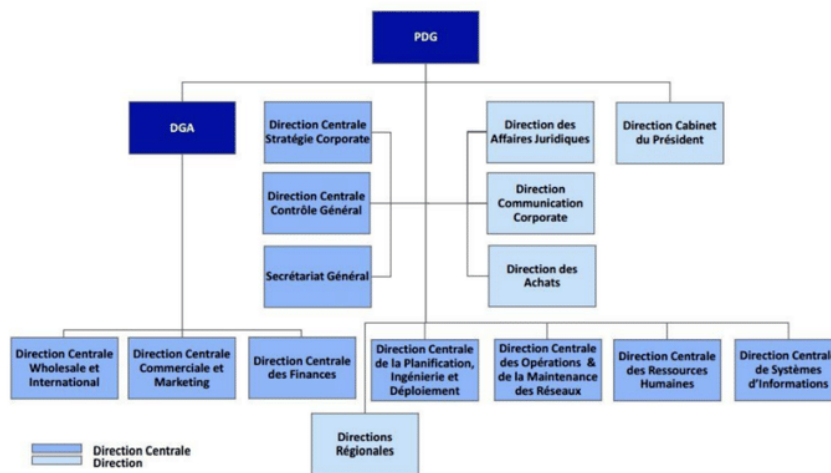


Fig. 1.2 : Organigramme de Tunisie Telecom

L'organigramme fait apparaître, sous la direction des infrastructures réseau, le service technique chargé de la sécurité des systèmes de communication — service au sein duquel ce projet a été réalisé. Ce positionnement organisationnel est déterminant : il confère au service une vision transversale sur les couches réseau, applicatives et sécurité, indispensable pour un projet de sécurisation IoT de bout en bout. Ses principales activités incluent :

- la gestion et l'administration des infrastructures réseau ;
- la supervision du trafic et des performances des systèmes ;
- la mise en œuvre de solutions de cybersécurité ;
- l'analyse des incidents et des menaces de sécurité ;
- l'évaluation et l'intégration de nouvelles technologies.

Dans ce cadre, le service s'intéresse particulièrement aux problématiques de sécurité liées aux architectures modernes telles que les environnements cloud, les réseaux virtualisés et les écosystèmes IoT. Le stage réalisé dans ce service offre ainsi l'opportunité de

travailler sur des problématiques concrètes liées à la sécurisation des communications dans les systèmes IoT.

1.2.2 Positionnement de Tunisie Telecom face aux enjeux IoT

Dans le cadre de sa stratégie de transformation numérique, *Tunisie Telecom* développe progressivement des offres dans le domaine de l’IoT et des services connectés. L’opérateur est confronté aux mêmes impératifs de sécurisation que ses homologues internationaux, dans un contexte supplémentaire de contraintes réglementaires nationales et de ressources techniques à mobiliser de manière optimale.

Le service d’accueil est particulièrement sensibilisé aux risques liés à l’interconnexion de dispositifs IoT sur les infrastructures de l’opérateur : hétérogénéité des équipements connectés, absence de standards unifiés d’authentification à l’échelle du parc, difficultés de supervision des flux applicatifs de type MQTT et manque d’outils permettant la détection de comportements anormaux au niveau applicatif. Ces constats constituent le point de départ direct du projet et justifient la construction d’un démonstrateur expérimental permettant de valider une approche intégrée avant tout passage à l’échelle opérationnel.

Ce positionnement stratégique de Tunisie Telecom fournit le contexte institutionnel du projet. Il convient maintenant de situer ce travail dans le cadre plus large du marché IoT et des enjeux propres aux opérateurs télécom.

1.3 Cadre général du projet

Dans le cadre de ce travail, l’objectif est de construire un prototype expérimental capable de sécuriser les flux de communication d’un écosystème IoT émulé. Le projet s’appuie sur une architecture représentative d’un environnement opérateur : segmentation réseau, services de confiance, broker MQTT, supervision de sécurité et analyse comportementale. Cette approche permet de valider les mécanismes proposés avant toute projection vers un déploiement réel à plus grande échelle. L’architecture repose sur cinq zones réseau isolées (IoT, DMZ, PKI, SIEM, Analyse), un broker MQTT au centre des flux applicatifs et des couches successives de protection — PKI, mTLS, IDS, SIEM, ML — qui matérialisent une stratégie de défense en profondeur. La représentation détaillée de cette architecture fait l’objet du chapitre 3 (figure 3.3) ; chaque chapitre ultérieur en détaille la conception puis la réalisation.

1.4 L’IoT dans les réseaux de télécommunications

1.4.1 Croissance et enjeux du marché IoT

L’Internet des Objets constitue l’une des évolutions technologiques les plus structurantes de la décennie actuelle. Selon l’ENISA [2], le nombre de dispositifs connectés actifs dans le monde poursuit une croissance soutenue, portée par la multiplication

des cas d’usage dans l’industrie, les villes intelligentes, la santé connectée, les infrastructures énergétiques et les réseaux de transport. Cette expansion génère un volume massif de données dont la collecte, le transport et le traitement reposent en très grande partie sur les infrastructures des opérateurs télécom.

Cette croissance s’accompagne cependant d’une complexification notable des exigences de sécurité. Meneghello et al. [3] soulignent que les dispositifs IoT présentent structurellement des vulnérabilités liées à leur conception : ressources processeur et mémoire limitées rendant difficile l’intégration de mécanismes cryptographiques complets, cycles de mise à jour longs ou inexistant, et exposition à des réseaux partagés peu maîtrisés. Ces fragilités sont exploitées de manière croissante par des acteurs malveillants, comme l’illustre l’attaque Mirai [4], qui a mobilisé des centaines de milliers d’objets compromis pour générer des attaques par déni de service dépassant 1 Tbit/s (analysée à la section 1.5).

1.4.2 Rôle des opérateurs télécom dans l’écosystème IoT

Les opérateurs télécom fournissent la connectivité (4G/5G, NB-IoT, LoRaWAN), hébergent les plateformes de gestion et proposent des services de sécurité managée. Ce positionnement crée des responsabilités spécifiques :

- **Multi-tenance** : isolation rigoureuse des flux entre clients verticaux distincts (énergie, transport, santé) ;
- **Volumétrie** : les mécanismes de sécurité doivent fonctionner à l’échelle de millions de publications/heure sans dégrader la latence ;
- **Hétérogénéité** : politiques de sécurité cohérentes pour des dispositifs aux capacités très disparates ;
- **Responsabilité réglementaire** : obligation de démontrer un niveau de sécurité maîtrisé et auditable.

Ces contraintes font de la sécurisation IoT un enjeu de premier ordre pour les opérateurs. Il convient de caractériser les menaces, puis de recenser les technologies de sécurisation disponibles.

1.5 Menaces et vulnérabilités spécifiques à l’IoT

1.5.1 Surface d’attaque étendue

Les écosystèmes IoT présentent une surface d’attaque significativement plus large que les systèmes informatiques classiques. Meneghello *et al.* [3] en distinguent quatre couches : (i) la couche *matérielle* (composants embarqués, interfaces de débogage exposées, attaques par canal auxiliaire) ; (ii) la couche *firmware* (mises à jour non signées,

exécution de code arbitraire); (iii) la couche *réseau* (transports en clair, protocoles propriétaires faibles); (iv) la couche *services et écosystème* (API non authentifiées, applications mobiles vulnérables, multi-tenance mal cloisonnée).

Au niveau matériel, les interfaces de débogage non désactivées (JTAG, UART) exposent un accès physique permettant l’extraction de clés cryptographiques ou la modification du firmware. Au niveau logiciel, les API REST ou MQTT de gestion des dispositifs constituent un vecteur complémentaire lorsqu’elles sont déployées sans authentification ou avec des mécanismes d’autorisation insuffisants. Ces vecteurs illustrent la nécessité d’une approche de sécurisation multicouche.

1.5.2 OWASP IoT Top 10 et menaces réseau

L’OWASP synthétise dans son *IoT Top 10* [5] les dix risques applicatifs majeurs : mots de passe faibles ou par défaut, services réseau non sécurisés, interfaces écosystème non protégées, mécanisme de mise à jour absent, composants obsolètes, protection insuffisante de la vie privée, transferts et stockage non chiffrés, absence de gestion des dispositifs, paramètres par défaut non sûrs, absence de hardening physique. Dans ce projet, nous nous concentrons sur les menaces **réseau** et **applicatives** : interception (*sniffing*), *man-in-the-middle*, force brute, exfiltration applicative, déni de service, vol/abus de certificats.

1.5.3 Études de cas de référence

L’analyse de l’attaque **Mirai** [4], [6] illustre toutes ces faiblesses simultanément : le botnet exploitait des couples identifiant/mot de passe par défaut sur des objets exposés en SSH/Telnet, recrutait des centaines de milliers de dispositifs en quelques jours, et générait des attaques DDoS dépassant 1 Tbit/s. Le rapport *ENISA Threat Landscape for IoT* [2] confirme que ces vecteurs restent dominants et identifie comme contre-mesures prioritaires : l’authentification forte, le chiffrement systématique, la segmentation réseau et la supervision active. Ces orientations rejoignent les recommandations du NIST [7] pour la cybersécurité des dispositifs IoT.

Cette cartographie des menaces permet d’identifier les vecteurs prioritaires à contrer : interception des flux, usurpation d’identité, déni de service et abus de certificats. La section suivante présente les technologies et protocoles mobilisés pour y répondre.

1.6 Technologies et protocoles de sécurisation

Le choix des technologies de sécurisation découle directement des menaces identifiées. Cette section suit un fil directeur continu : du protocole applicatif au chiffrement du transport, de l’authentification mutuelle à la PKI, de la détection par règles à l’apprentissage automatique.

1.6.1 Protocoles applicatifs IoT

Quatre protocoles représentatifs ont été comparés (tableau 1.1) :

Tab. 1.1 : Comparaison des principaux protocoles applicatifs IoT

Critère	MQTT 5	CoAP	AMQP 1.0	HTTP/REST
Modèle	Pub/Sub	Req/Rep	Pub/Sub + queues	Req/Rep
Transport	TCP/TLS	UDP/DTLS	TCP/TLS	TCP/TLS
Empreinte	Très faible	Très faible	Moyenne	Moyenne/élevée
QoS	0/1/2	Confirmable	0/1	Aucun natif
Sécurité native	TLS+ACL	DTLS+OSCORE	SASL+TLS	TLS+JWT
Maturité IoT	Très large	Large	Industrielle	Universelle

MQTT 5 [8] est retenu pour son empreinte faible, sa large adoption industrielle et ses mécanismes natifs TLS+ACL combinables avec le mTLS. MQTT ne garantissant ni confidentialité ni authenticité, TLS 1.3 avec authentification mutuelle s’impose.

1.6.2 Sécurisation du transport : TLS 1.3 et mTLS

1.6.2.1 TLS 1.3

Le protocole TLS 1.3 [9] apporte par rapport à TLS 1.2 une simplification significative : suppression des suites cryptographiques obsolètes, négociation en **1-RTT** (et 0-RTT optionnel), *forward secrecy* obligatoire, signatures et échanges de clés modernisés (ECDHE, EdDSA, AES-GCM, ChaCha20-Poly1305). Les recommandations opérationnelles publiées dans la RFC 9325 [10] guident le choix des paramètres et la durée de vie des certificats.

Par ailleurs, le protocole **DTLS** (*Datagram Transport Layer Security*) constitue l’adaptation de TLS aux transports non fiables, principalement UDP. Il intègre des mécanismes de retransmission et de numérotation de séquence et sert à sécuriser le protocole CoAP, privilégié pour les dispositifs à très faibles ressources. Dans le cadre de ce projet, le choix de MQTT (fondé sur TCP) dicte l’utilisation de TLS plutôt que de DTLS.

1.6.2.2 Authentification mutuelle

Dans un déploiement classique, seul le serveur s’authentifie auprès du client. Le **mTLS** (*mutual TLS*) impose en outre au client de présenter un certificat X.509 valide signé par une autorité de confiance. Cette double authentification permet : (i) d’éviter qu’un objet illégitime se connecte au broker ; (ii) de chiffrer chaque session de bout en bout ; (iii) de tracer chaque message via l’identité cryptographique de son émetteur. Cette propriété est essentielle pour la mise en place ultérieure d’ACL par certificat dans Mosquitto [11].

Resultat

Positionnement critique : TLS 1.3 + mTLS TLS 1.3 supprime les suites vulnérables, négocie en 1-RTT et impose la *forward secrecy*. Le mTLS substitue à l’authentification par mot de passe une preuve cryptographique d’identité non duplicable. Cette double garantie – confidentialité *et* authenticité mutuelle – ferme simultanément les vecteurs d’interception et d’usurpation identifiés.

L’authentification mutuelle par certificats suppose cependant l’existence d’une infrastructure capable d’**émettre, renouveler et révoquer** ces certificats à l’échelle d’un parc de dispositifs : c’est précisément le rôle d’une PKI automatisée.

1.6.3 Gestion des identités : PKI et automatisation

Une PKI à deux niveaux (CA racine *offline* + CA intermédiaire *online*) constitue la pratique recommandée [7] : la CA racine ne signe que des CA intermédiaires et reste hors ligne ; les CA intermédiaires signent les certificats opérationnels (services, dispositifs). En cas de compromission d’une CA intermédiaire, seul le sous-arbre concerné doit être révoqué.

L’automatisation de l’émission est indispensable lorsque le parc dépasse quelques dizaines d’objets. **HashiCorp Vault** [12] fournit un *secrets engine pki* qui expose une API HTTP de demande de certificats avec contrôle fin (rôles, ACL, TTL). C’est la solution retenue dans ce projet.

Resultat

Positionnement critique : Vault vs alternatives PKI OpenSSL CLI est entièrement manuel, EJBCA est disproportionné pour un projet de quatre mois, Let’s Encrypt/ACME est inadapté à une PKI interne IoT. **Vault** est retenu car il réunit : API REST automatisée, gestion fine des rôles/TTL, intégration DevOps et déploiement on-premise – critère non négociable pour un opérateur soucieux de souveraineté.

Même avec une PKI robuste et un chiffrement bout en bout, il n’est pas possible d’exclure tout incident : un certificat peut être volé, un dispositif légitime peut adopter un comportement déviant, ou une configuration insuffisante peut ouvrir un vecteur non anticipé. La **supervision active** – combinant IDS réseau et SIEM – constitue le filet de sécurité complémentaire à ces mécanismes préventifs.

1.6.4 Détection d’intrusion et supervision

1.6.4.1 IDS réseau orienté MQTT

Suricata est un IDS/IPS open source qui dispose, depuis la version 6, d’un *parser* natif MQTT capable d’extraire les champs **CONNECT**, **PUBLISH**, **SUBSCRIBE** [13]. Cette capacité permet de définir des règles métier (par exemple : alerter sur un **CONNECT** sans **ClientID** reconnu, ou sur une rafale de **CONNECT** indiquant un *brute force*) qui ne seraient pas accessibles à un IDS générique.

Le tableau 1.2 compare trois solutions IDS open source représentatives sur les critères déterminants pour ce projet.

Tab. 1.2 : Comparaison des solutions IDS open source

Critère	Suricata 6	Snort 3	Zeek 6
Parser MQTT natif	Oui (v6+)	Non	Partiel (scripts)
IDS/IPS actif	Oui	Oui	Non (analyse seule)
Multi-threading	Oui (natif)	Oui (v3)	Oui
Intégration Wazuh	Native (eve.json)	Adaptateur requis	Scripts Lua
Règles Sigma/Suricata	Native	Conversion requise	Non
Maturité IoT	Élevée	Moyenne	Élevée (analytique)
Verdict	Retenu	Non retenu	Complémentaire

Suricata est retenu pour son parser MQTT natif, son format `eve.json` nativement consommé par Wazuh, et son moteur multi-thread.

1.6.4.2 SIEM Wazuh

Wazuh [14] est une plateforme de *Security Information and Event Management* qui agrège les journaux d’agents endpoint, de sources réseau (Suricata) et applicatifs (Vault, Mosquitto). Elle s’intègre nativement à OpenSearch pour le stockage et au tableau de bord Wazuh Dashboard pour la visualisation et la corrélation. Sa modularité (règles XML, décodeurs personnalisables) en fait un candidat naturel pour la consolidation des alertes IoT.

Le tableau 1.3 compare Wazuh aux deux autres plates-formes SIEM courantes sur les critères déterminants pour un projet IoT académique à contraintes de coût et de souveraineté.

Wazuh est retenu pour son intégration native avec Suricata (`eve.json`), ses agents endpoint, OpenSearch et sa licence open source garantissant la souveraineté des données.

Les règles IDS ne couvrent que les attaques connues ; une couche ML est nécessaire pour les comportements nouveaux.

Tab. 1.3 : Comparaison des solutions SIEM

Critère	Wazuh 4.7	ELK Stack	Splunk Free
Licence	Open source (OSS)	Open source	Propriétaire (limité)
Intégration Suricata	Native (eve.json)	Via Filebeat	Via add-on HEC
Corrélation native	Oui (règles XML)	Non (Elastic SIEM en payant)	Oui (SPL)
Agents endpoint	Oui (Wazuh agent)	Non natif	Oui (UF)
Volume journaux (gratuit)	Illimité	Illimité	500 MB/j
Courbe d’apprentissage	Modérée	Élevée (tuning)	Modérée
Souveraineté données	Totale (on-premise)	Totale	Partielle (cloud)
Verdict	Retenu	Non retenu	Non retenu

1.6.5 Apprentissage automatique pour la détection d’anomalies

Les approches par règles statiques sont efficaces pour les attaques connues mais peinent à détecter les comportements nouveaux. Hasan *et al.* [15] ont montré qu’un classifieur supervisé entraîné sur des features réseau et applicatives obtient des taux de détection supérieurs à 99 % sur sept catégories d’attaques IoT. Les algorithmes les plus utilisés sont :

- **Isolation Forest** [16] : méthode non supervisée d’isolement, particulièrement efficace en haute dimension et adaptée aux scénarios où les anomalies sont rares ;
- **Random Forest** [17] : ensemble d’arbres de décision, robuste, interprétable (*feature importance*) et performant en classification multi-classes.

Ces deux algorithmes sont complémentaires : IF (non supervisé) modélise la normalité et détecte les anomalies inconnues (scénario S6) ; RF (supervisé) fournit une classification précise en sept catégories avec mesure d’importance par feature. La bibliothèque *scikit-learn* [18] constitue le socle du pipeline ML.

1.7 Étude de l’existant et analyse critique

Les technologies étant recensées, il est indispensable de dresser un état des lieux objectif des pratiques de sécurité IoT et d’analyser les solutions existantes, afin de quantifier les lacunes que la solution proposée doit combler. Ce diagnostic s’appuie sur l’analyse des déploiements typiques, les recommandations publiées par l’ENISA [2] et le NIST [7], ainsi que sur les observations réalisées dans le cadre du stage chez *Tunisie Telecom*.

1.7.1 État des lieux de la sécurité IoT dans les déploiements opérateur

L'analyse conduite en amont met en évidence six lacunes récurrentes :

- **Transport non chiffré** : MQTT souvent déployé sur le port 1883 en clair ;
- **Authentification absente ou unidirectionnelle** : pas de vérification du certificat client, mots de passe faibles ;
- **Absence de PKI dédiée** : certificats auto-signés, non révocables, sans cycle de vie ;
- **Contrôle d'accès insuffisant** : ACL MQTT rarement mises en œuvre ;
- **Supervision fragmentée** : outils réseau et applicatifs non corrélés ;
- **Absence de détection comportementale** : IDS limités aux signatures connues.

1.7.2 Analyse critique des solutions existantes

Trois familles d'approches ont été évaluées : les **plateformes cloud commerciales** (AWS IoT, Azure IoT Hub), qui offrent un déploiement rapide mais induisent un verrouillage fournisseur incompatible avec la souveraineté opérateur ; les **distributions open source intégrées** (ThingsBoard, Eclipse Kura), qui ne proposent ni mTLS systématique, ni PKI automatisée, ni corrélation IDS/ML ; et l'**architecture sur mesure** à base de composants open source, qui garantit le contrôle total au prix d'une complexité initiale. L'analyse fait en outre ressortir une faible intégration entre couches réseau et applicative, et une reproductibilité limitée des architectures publiées. Pour un opérateur soucieux de souveraineté et de maîtrise, l'architecture sur mesure s'impose ; c'est le positionnement retenu dans ce projet.

1.7.3 Synthèse du diagnostic

Le tableau 1.4 synthétise le diagnostic en mettant en regard chaque lacune identifiée avec la brique de sécurité retenue dans la solution, établissant ainsi le lien direct entre le contexte opérationnel observé et les choix de conception du projet.

Ces lacunes exigent une réponse architecturale intégrée. La formulation d'une problématique globale s'impose.

Tab. 1.4 : Diagnostic des lacunes et briques de réponse correspondantes

Lacune identifiée	Brique de réponse retenue
Flux MQTT en clair, sans chiffrement de bout en bout	Activation obligatoire de TLS 1.3 sur Mosquitto (port 8883)
Authentification client absente ou basée sur mot de passe faible	Certificat X.509 client obligatoire, vérifié par le broker en mTLS
Gestion manuelle et non automatisée des certificats	PKI HashiCorp Vault à deux niveaux avec émission et révocation via API
Accès applicatifs non restreints par dispositif	ACL Mosquitto fondées sur le <i>Common Name</i> du certificat client
Supervision réseau et applicative déconnectées	Intégration de Suricata au SIEM Wazuh pour la corrélation
Absence de détection comportementale	Module ML fondé sur IsolationForest et Random Forest

1.8 Problématique

L'étude de l'existant a mis en lumière des lacunes structurelles interdépendantes dans la sécurisation des flux IoT opérateur. L'**hétérogénéité** des dispositifs (microcontrôleurs, passerelles, modems cellulaires) et des protocoles, la **volumétrie** massive et asynchrone des messages, les **contraintes embarquées** (ressources limitées, alimentation batterie, connectivité variable), la **multi-tenance** imposant une segmentation rigoureuse des flux clients, et la **gestion à grande échelle des identités cryptographiques** (émission, renouvellement et révocation de certificats X.509 pour des dizaines de milliers de dispositifs) forment une surface d'attaque étendue que des correctifs ponctuels ne peuvent couvrir. La problématique se formule ainsi :

Problématique

Comment concevoir, déployer et valider, dans un environnement émulé représentatif d'un opérateur télécom, une architecture de sécurisation **de bout en bout** des flux de communication d'un écosystème IoT, qui combine : (i) un chiffrement et une authentification mutuelle systématiques entre objets et infrastructures ; (ii) une gestion automatisée du cycle de vie des certificats ; (iii) une supervision active des menaces par corrélation d'alertes réseau et apprentissage automatique ; (iv) tout en restant compatible avec les contraintes de performance et de scalabilité du domaine ?

Pour répondre à cette problématique, un objectif général et des objectifs spécifiques doivent être clairement définis, afin de structurer l'ensemble des travaux à mener.

1.9 Objectifs du projet

1.9.1 Objectif général

L’objectif général est de concevoir, implémenter et valider une **architecture de sécurisation de bout en bout** des flux IoT, fondée sur la **défense en profondeur** : TLS 1.3 + mTLS, PKI automatisée, ACL par certificat, IDS Suricata, SIEM Wazuh et détection ML — exclusivement en composants open source et souverains. Cet objectif se décline en sept objectifs spécifiques :

1.9.2 Objectifs spécifiques

Les sept objectifs spécifiques (O1–O7) structurent l’ensemble des travaux et sont présentés dans le tableau 1.5 :

Tab. 1.5 : Objectifs spécifiques du projet

ID	Objectif
O1	Concevoir une architecture IoT segmentée et reproductible, déployable dans un environnement émulé.
O2	Déployer une infrastructure à clés publiques (PKI) automatisée fondée sur HashiCorp Vault.
O3	Imposer un chiffrement TLS 1.3 avec authentification mutuelle (mTLS) sur le broker MQTT.
O4	Émuler une flotte hétérogène de 50 dispositifs IoT et générer un trafic réaliste mêlant comportements légitimes et attaques.
O5	Mettre en place une analyse réseau native MQTT au moyen d’un IDS Suricata.
O6	Centraliser et corrélérer les événements de sécurité dans un SIEM Wazuh, avec visualisation via OpenSearch Dashboards.
O7	Développer un module de détection d’anomalies par apprentissage automatique (IsolationForest et Random Forest) et évaluer son apport.

Ces objectifs traduisent opérationnellement la problématique et structurent les différentes phases du projet. Avant de délimiter le périmètre du travail, il convient de préciser les besoins fonctionnels auxquels la solution doit répondre.

1.10 Besoins fonctionnels

Six domaines fonctionnels ont été identifiés : PKI automatisée, TLS 1.3 + mTLS, ACL par certificat, simulation du trafic, détection IDS/SIEM et détection ML. Leur spécification formelle figure au chapitre 3 (tableaux 3.1 et 3.2).

1.11 Périmètre et contraintes du projet

1.11.1 Périmètre fonctionnel

Le tableau 1.6 précise le périmètre du projet :

Tab. 1.6 : Périmètre du projet – inclus et exclus

Inclus dans le périmètre	Exclu du périmètre
Segmentation réseau par VLANs sous GNS3	Déploiement en environnement de production réel
PKI à deux niveaux avec HashiCorp Vault	Gestion des mises à jour firmware des objets (OTA)
Broker MQTT en TLS 1.3 + mTLS avec ACL par topic	Sécurité de la couche matérielle des dispositifs
Émulation d’une flotte de 50 dispositifs hétérogènes	Flotte supérieure à 50 dispositifs (contrainte ressources hôte)
Détection IDS Suricata + corrélation SIEM Wazuh	Intégration aux outils SOC existants de Tunisie Telecom
Pipeline ML hors ligne (IsolationForest, Random Forest)	Scoring ML en temps réel sur flux de données en streaming
Évaluation des performances (latence, débit mTLS)	Tests de charge en environnement radio réel (4G/5G/NB-IoT)

1.11.2 Contraintes du projet

Quatre contraintes ont influencé directement les choix architecturaux : (i) **matérielle** — station Windows 10/11 avec 16 Go de RAM, sans cloud dédié ni équipements physiques, délimitant la taille de la flotte ; (ii) **temporelle** — stage de quatre mois imposant une priorisation Scrum des livrables ; (iii) **reproductibilité** — environnement reconstituable depuis zéro à partir des scripts versionnés ; (iv) **souveraineté** — composants exclusivement open source.

1.11.3 Hypothèses de travail

L’expérimentation repose sur trois hypothèses : (1) le dataset de 76 000 messages en huit classes [15] est représentatif du trafic IoT réel ; (2) les mesures émulées (latence mTLS, débit) constituent des ordres de grandeur valides, sans reproduire un réseau radio réel ; (3) la détection par règles (Suricata/Wazuh) et par ML sont des couches complémentaires et non substituables.

1.12 Démarche méthodologique

Trois cadres agiles ont été évalués : Scrum [19], Kanban [20] et XP [21]. **Scrum** a été retenu pour trois raisons : (i) l’alignement des revues de sprint avec les jalons académiques (rapport intermédiaire, soutenance) ; (ii) la flexibilité du backlog face aux imprévus techniques (blocage réseau, problèmes de certificats) ; (iii) la priorisation garantissant la livraison des fonctionnalités critiques en premier. Kanban a été écarté pour son absence de cadence fixe, incompatible avec des jalons imposés, et XP pour ses pratiques (TDD, *pair-programming*) disproportionnées en développement solo. Six sprints de deux semaines structurent la réalisation ; leur planification figure au chapitre 3.

1.13 Conclusion

Ce chapitre a établi le cadre de référence du projet : contexte institutionnel, menaces IoT, technologies de sécurisation, diagnostic de l’existant, problématique, objectifs et périmètre. Le fil directeur est continu : des menaces découlent les exigences protocolaires, du choix de MQTT découle le besoin de TLS 1.3/mTLS, puis d’une PKI automatisée, puis d’une supervision IDS/SIEM, puis de l’apport ML. Le projet se positionne comme une **architecture de référence open source** pour la sécurisation IoT opérateur. Le chapitre suivant présente l’environnement technique du laboratoire.

Chapitre 2

Environnement technique du laboratoire

2.1 Introduction

Ce chapitre présente concrètement l'environnement expérimental : chaque composant du laboratoire GNS3, son rôle, son image de base, sa position réseau et ses interfaces.

2.2 Vue d'ensemble du laboratoire

L'ensemble du laboratoire est émulé sous **GNS3** [22], qui exécute les systèmes d'exploitation et services réels (VM QEMU + conteneurs Docker). La topologie repose sur cinq zones réseau isolées par VLANs, reliées par un routeur MikroTik CHR et protégées par un pare-feu pfSense.

Le tableau 2.1 présente l'inventaire complet des nœuds du laboratoire.

La figure 2.1 présente la topologie complète du laboratoire avec les cinq zones réseau et les interactions principales entre les composants.

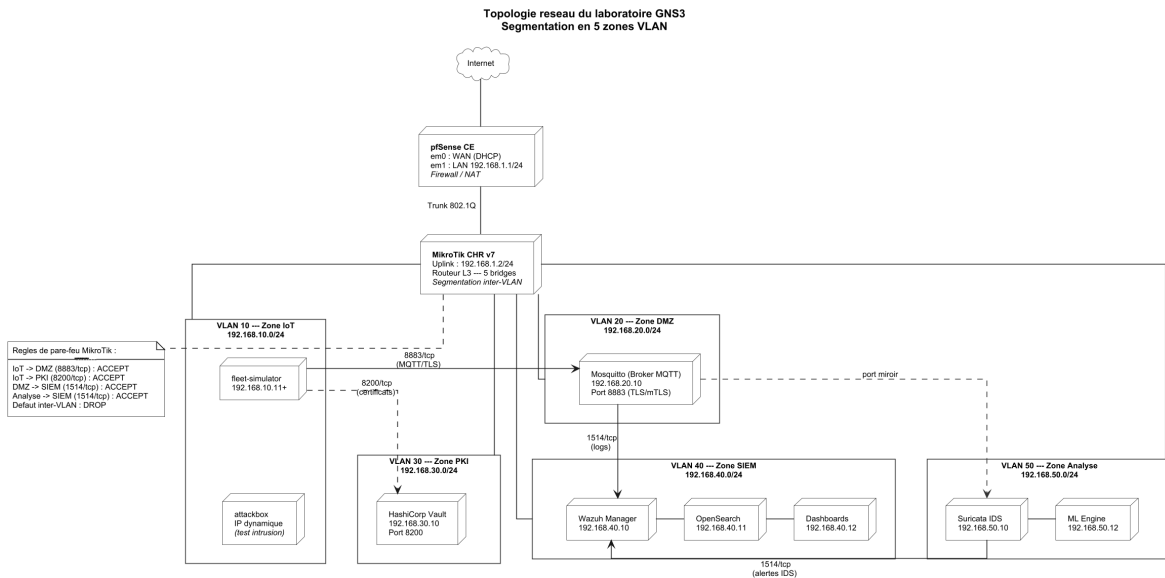


Fig. 2.1 : Topologie du laboratoire GNS3

Tab. 2.1 : Inventaire des nœuds du laboratoire GNS3

Nœud	Image	Zone	IP	Port	Rôle
pfSense	CE 2.7.2	WAN/LAN	192.168.1.1	–	Pare-feu, NAT
MikroTik	CHR 7.16	Trunk	.x.1	–	Routeur L3, VLANs
fleet-sim	python :3.11-alpine	IoT (10)	.10.11+	–	Simulateur de flotte
mosquitto	eclipse-mosquitto :2.0	DMZ (20)	.20.10	8883	Broker MQTT mTLS
vault	hashicorp/vault :1.15.4	PKI (30)	.30.10	8200	PKI, CA, certificats
wazuh	wazuh/manager :4.7	SIEM (40)	.40.10	1514	SIEM, corrélation
opensearch	opensearch :2.x	SIEM (40)	.40.11	9200	Indexation des événements
dashboards	opensearch-dashboards	SIEM (40)	.40.12	5601	Visualisation, tableaux
suricata	–	Analyse (50)	.50.10	–	IDS réseau, parser MQTT
ml-engine	python :3.11-slim	Analyse (50)	.50.12	5000	Détection d'anomalies ML
attackbox	python :3.11-alpine	IoT (10)	dynamique	–	Boîte à outils d'attaque

Le schéma met en évidence l'isolement strict entre zones : les dispositifs IoT n'atteignent que le broker (DMZ) et la PKI, tandis que les flux de supervision convergent vers la zone SIEM. Cette séparation matérialise le principe *deny-by-default* au cœur de l'architecture.

La figure 2.2 présente la topologie réelle dans GNS3.

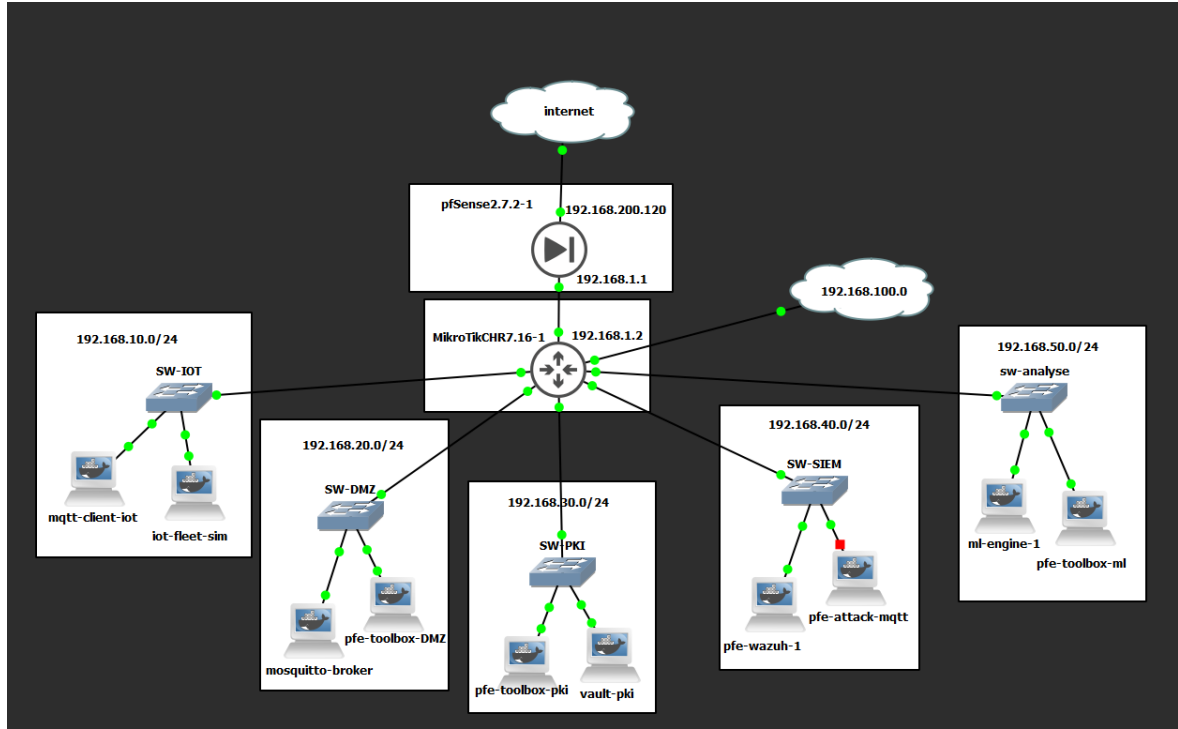


Fig. 2.2 : Topologie réelle dans GNS3

La correspondance entre la topologie logique et son implémentation effective dans GNS3 atteste de la reproductibilité de l'environnement : chaque nœud, chaque lien et chaque zone du schéma conceptuel se retrouve dans la capture, confirmant que le laboratoire peut être reconstruit à l'identique à partir des scripts versionnés. Les sections suivantes décrivent chaque composant en détail.

2.3 Infrastructure réseau

2.3.1 pfSense — pare-feu et passerelle WAN

pfSense Community Edition 2.7.2 est un pare-feu open source basé sur FreeBSD, déployé en tant que machine virtuelle QEMU dans GNS3. Il constitue le point d'entrée du laboratoire vers Internet et assure quatre fonctions :

- **NAT et routage WAN** : l'interface `em0` obtient une adresse IP par DHCP depuis le réseau hôte et fournit l'accès Internet à l'ensemble du laboratoire via NAT sortant pour chacun des cinq sous-réseaux ;
- **Filtrage périmétrique** : pfSense applique les règles de filtrage entre le réseau externe et le réseau interne (192.168.1.0/24), bloquant tout trafic entrant non sollicité ;

- **Filtrage secondaire LAN** : sur son interface LAN, pfSense applique des règles complémentaires par couple source/destination (par exemple IoT→Broker/8883, DMZ→Wazuh/1514, blocage IoT→SIEM et IoT→Analyse), formant un **second niveau de filtrage** qui double la politique inter-VLAN du MikroTik ;
- **Liaison vers le routeur interne** : l'interface **em1** (192.168.1.1/24) est connectée au MikroTik, qui prend en charge la segmentation interne en VLANs.

2.3.2 MikroTik CHR — routeur L3 et segmentation VLAN

MikroTik Cloud Hosted Router (CHR) 7.16 est un routeur logiciel déployé en tant que VM QEMU. Il constitue le cœur de la segmentation réseau du laboratoire. Sa configuration repose sur cinq *bridges*, chacun associé à une zone réseau et à une interface physique distincte :

- **bridge-iot** (ether2) : zone IoT, passerelle 192.168.10.1/24 ;
- **bridge-dmz** (ether3) : zone DMZ, passerelle 192.168.20.1/24 ;
- **bridge-pki** (ether4) : zone PKI, passerelle 192.168.30.1/24 ;
- **bridge-siem** (ether5) : zone SIEM, passerelle 192.168.40.1/24 ;
- **bridge-analyse** (ether6) : zone Analyse, passerelle 192.168.50.1/24.

Le MikroTik applique une politique de filtrage inter-VLAN en mode *deny-by-default* : seuls les flux explicitement autorisés traversent les frontières de zone. Il fournit également la fonction de **port mirroring**, copiant passivement le trafic de la zone DMZ vers Suricata dans la zone Analyse, sans altérer les paquets en transit.

2.3.3 Politique de sécurité inter-zones

Le tableau 2.2 résume les règles de filtrage inter-VLAN appliquées par le MikroTik. Tout flux non listé est bloqué.

Cette politique garantit que les dispositifs IoT ne peuvent communiquer qu'avec le broker et la PKI, empêchant toute interaction directe avec les systèmes de supervision ou d'analyse.

Tab. 2.2 : Politique de filtrage inter-VLAN (*deny-by-default*)

Source	Destination	Proto/Port	Action	Justification
IoT (10)	DMZ .20.10	TCP/8883	Accept	MQTT/TLS vers broker
IoT (10)	PKI .30.10	TCP/8200	Accept	Demande de certificat
DMZ (20)	PKI .30.10	TCP/8200	Accept	Vérification de certificat
DMZ (20)	SIEM .40.10	TCP/1514	Accept	Envoi des journaux
Analyse (50)	SIEM .40.10	TCP/1514	Accept	Envoi des alertes
SIEM (40)	SIEM (40)	Tous	Accept	Communications internes
Tous	SIEM .40.12	TCP/5601	Accept	Accès aux tableaux de bord
IoT (10)	SIEM (40)	–	Drop	Isolation directe
IoT (10)	Analyse (50)	–	Drop	Isolation directe
Autre	Autre	–	Drop	Politique par défaut

2.4 Composants applicatifs

Chaque composant applicatif est déployé sous forme de conteneur Docker intégré à GNS3. Les sous-sections suivantes décrivent le rôle, le fonctionnement et les interfaces de chacun.

2.4.1 Broker MQTT Mosquitto (zone DMZ)

Eclipse Mosquitto 2.0 [11] est le broker MQTT central de l'architecture. Construit à partir de l'image officielle `eclipse-mosquitto:2.0`, il constitue le point de passage obligatoire de tous les flux applicatifs IoT.

Rôle. Mosquitto reçoit les publications des 50 dispositifs émulés, gère les souscriptions et applique le contrôle d'accès. Il est le seul service directement accessible depuis la zone IoT.

Sécurisation. Le broker est configuré exclusivement en mode TLS 1.3 sur le port 8883. L'authentification anonyme est désactivée ; chaque client doit présenter un certificat X.509 signé par la CA intermédiaire (`require_certificate true`). Le *Common Name* (CN) du certificat est utilisé comme identifiant MQTT (`use_identity_as_username true`), ce qui permet d'appliquer des **ACL par dispositif** : chaque objet ne peut publier que sur ses propres topics (`iot/+/<device_id>/#`) et lire uniquement ses commandes (`iot/commands/<device_id>/#`).

Journalisation. Un agent **rsyslog** embarqué dans le conteneur surveille le fichier de log Mosquitto et transmet les événements au SIEM Wazuh (192.168.40.10, port UDP 514) en temps réel. Ce mécanisme permet la corrélation des événements MQTT avec les alertes réseau de Suricata.

Interfaces.

- **Entrée** : flux MQTT/TLS (TCP/8883) depuis la zone IoT et le ML Engine ;
- **Sortie** : journaux Syslog (UDP/514) vers Wazuh ; vérification de certificats (TCP/8200) vers Vault.

2.4.2 HashiCorp Vault (zone PKI)

HashiCorp Vault 1.15.4 [12] est le serveur de gestion des identités cryptographiques. Déployé à partir de l'image officielle `hashicorp/vault:1.15.4`, il héberge l'infrastructure à clés publiques (PKI) du laboratoire.

Rôle. Vault fournit une PKI automatisée à deux niveaux : une **CA racine** (*offline*, clé RSA 4096 bits, TTL 10 ans) qui ne signe que la CA intermédiaire, et une **CA intermédiaire** (*online*, TTL 5 ans) qui émet les certificats opérationnels. L'API REST de Vault (port 8200) permet l'émission, le renouvellement et la révocation automatisés des certificats, sans intervention manuelle.

Rôles PKI. Deux rôles sont configurés pour encadrer l'émission :

- **iot-services** : pour les services d'infrastructure (Mosquitto, Suricata, ML Engine). Domaines autorisés : `*.iot-pfe.local`, `*.dmz.iot-pfe.local`. TTL : 30 jours, max 1 an. Flags : serveur + client ;
- **iot-devices** : pour les dispositifs IoT. Domaines autorisés : `*.iot.iot-pfe.local`. TTL : 24 heures, max 30 jours. Flags : client uniquement.

Interfaces.

- **Entrée** : requêtes API REST (TCP/8200) depuis la zone IoT (demande de certificats) et la zone DMZ (vérification) ;
- **Stockage** : backend fichier local (`/vault/data`).

2.4.3 Simulateur de flotte IoT — fleet-sim (zone IoT)

Le **fleet-sim** est un simulateur de flotte développé en Python, construit sur l'image `python:3.11-alpine`. Il émule le comportement de 50 dispositifs IoT hétérogènes.

Rôle. Le simulateur rejoue un jeu de données de 76 000 messages issus d'un *dataset* de référence [15], couvrant huit catégories : trafic normal et sept types d'attaques. Chaque dispositif émulé établit une connexion MQTT/TLS 1.3 avec authentification mutuelle en utilisant son propre certificat client.

Fonctionnement. Pour chaque dispositif, le simulateur :

1. charge le certificat client correspondant (`certs/<device_id>/client.crt` et `client.key`) ;
2. établit un contexte TLS 1.3 avec vérification du certificat serveur (`ca-chain.crt`) ;
3. se connecte au broker Mosquitto (192.168.20.10:8883) avec le *ClientID* dérivé du *device_id* ;
4. publie les messages du dataset sur les topics MQTT structurés (`iot/<tenant>/<device>/<type>`) ;

Interfaces.

- **Sortie** : flux MQTT/TLS (TCP/8883) vers Mosquitto ;
- **Dépendance** : certificats émis par Vault.

2.4.4 Wazuh Manager (zone SIEM)

Wazuh Manager 4.7 [14] est le composant central de la supervision de sécurité. Il agrège les journaux provenant de toutes les zones et applique des règles de corrélation.

Rôle. Le manager reçoit les journaux de Mosquitto (via rsyslog), les alertes de Suricata (format `eve.json`) et les événements des agents endpoint déployés sur les conteneurs. Il applique des décodeurs et des règles XML pour identifier les événements de sécurité, générer des alertes et alimenter le tableau de bord.

Écosystème SIEM. La zone SIEM héberge trois composants complémentaires :

- **Wazuh Manager** (192.168.40.10, port 1514/1515) : collecte et corrélation ;
- **OpenSearch** (192.168.40.11, port 9200) : moteur d'indexation et de recherche des événements ;
- **OpenSearch Dashboards** (192.168.40.12, port 5601) : interface de visualisation accessible depuis toutes les zones, permettant la consultation des alertes et la construction de tableaux de bord.

Interfaces.

- **Entrée** : journaux Syslog (UDP/514 et TCP/1514) depuis DMZ et Analyse ; événements agents endpoint ;
- **Sortie** : indexation vers OpenSearch ; tableaux de bord via Dashboards.

2.4.5 OpenSearch (zone SIEM)

OpenSearch 2.x est le moteur d'indexation et de recherche déployé dans la zone SIEM (192.168.40.11, port 9200). Il reçoit les événements enrichis par Wazuh Manager et les stocke dans des index structurés, permettant des requêtes rapides sur de grands volumes d'alertes. OpenSearch constitue le backend de stockage de toute la chaîne de supervision : chaque alerte Suricata, chaque événement MQTT et chaque score ML y est indexé et consultable.

Interfaces.

- **Entrée** : événements indexés depuis Wazuh Manager (TCP/9200) ;
- **Sortie** : données consultables par OpenSearch Dashboards et par l'API REST.

2.4.6 OpenSearch Dashboards (zone SIEM)

OpenSearch Dashboards (192.168.40.12, port 5601) est l'interface de visualisation web de la pile SIEM. Accessible depuis toutes les zones du laboratoire (conformément à la politique de filtrage définie au tableau 2.2), il permet à l'opérateur SOC de consulter les alertes, de construire des tableaux de bord personnalisés et de corréler visuellement les événements par type d'attaque, par zone source ou par période.

Interfaces.

- **Entrée** : requêtes vers OpenSearch (TCP/9200) ;
- **Sortie** : interface web (TCP/5601) accessible depuis toutes les zones.

2.4.7 Suricata IDS (zone Analyse)

Suricata 7.0 [13] est l'IDS réseau du laboratoire. Il opère en mode passif sur un miroir du trafic de la zone DMZ.

Rôle. Suricata analyse le trafic réseau en temps réel à l'aide de règles de détection. Son *parser* MQTT natif (disponible depuis la version 6) lui permet d'inspecter les champs protocolaires MQTT : CONNECT, PUBLISH, SUBSCRIBE, DISCONNECT. Il peut ainsi détecter des comportements suspects au niveau applicatif, tels qu'une rafale de CONNECT (indicative d'un *brute force*) ou un PUBLISH sur un topic non autorisé.

Capture du trafic. Le MikroTik copie passivement le trafic de la zone DMZ (**bridge-dmz**) vers l'interface réseau de Suricata via la fonction de *port mirroring*. Ce mécanisme garantit que Suricata observe l'intégralité des échanges MQTT sans s'insérer dans le chemin de données, préservant ainsi la latence et la fiabilité du flux applicatif.

Interfaces.

- **Entrée** : trafic miroir de la zone DMZ (copie passive) ;
- **Sortie** : alertes au format `eve.json` transmises au SIEM Wazuh (TCP/1514).

2.4.8 Moteur ML — ml-engine (zone Analyse)

Le **ML Engine** est un service de détection d'anomalies développé en Python (image `python:3.11-slim`). Il constitue la couche de détection comportementale du laboratoire.

Rôle. Le moteur ML se connecte au broker Mosquitto en tant que client mTLS (avec le CN `ml.iot-pfe.local`) et souscrit à l'ensemble des topics IoT (`#`). Pour chaque message reçu, il construit un vecteur de 29 features — 21 numériques (taille du payload, entropie de Shannon, fréquence de publication, RTT, jitter, etc.) et 8 catégorielles encodées (protocole, version TLS, zone source/destination, etc.) — et le soumet à trois modèles pré-entraînés :

- **Random Forest binaire** : classification normal/attaque avec probabilité ;
- **Random Forest multi-classes** : identification du type d'attaque parmi sept catégories ;
- **Isolation Forest** : détection non supervisée des anomalies comportementales.

Les prédictions sont exposées via une **API REST** (Flask, port 5000) et les alertes sont transmises au SIEM Wazuh.

ACL Mosquitto. Le ML Engine dispose d'un accès en **lecture seule** sur l'ensemble des topics (`topic read #`), ce qui lui permet d'observer tout le trafic sans pouvoir publier de messages. Cette restriction est appliquée par l'ACL Mosquitto.

Interfaces.

- **Entrée** : souscription MQTT/TLS (TCP/8883) depuis Mosquitto ;
- **Sortie** : API REST (TCP/5000) pour la consultation des résultats ; alertes vers Wazuh (TCP/1514).

2.4.9 Boîte à outils d'attaque — attackbox (zone IoT)

La **boîte à outils d'attaque** est un conteneur Docker spécialisé (image `python:3.11-alpine`), déployé ponctuellement dans la zone IoT pour exécuter les scénarios d'attaque définis au chapitre 3.

Rôle. Ce conteneur fournit un environnement complet pour simuler les attaques ciblant l'écosystème IoT : outils réseau (`tcpdump`, `nmap`, `openssl`), bibliothèque MQTT Python (`paho-mqtt`) et scripts d'attaque spécialisés (flood MQTT, tentatives de connexion avec certificats invalides, exfiltration de données). Il dispose de certificats bootstrap permettant de tester aussi bien les connexions légitimes que les scénarios de rejet.

Interfaces.

- **Sortie** : flux d'attaque vers Mosquitto (TCP/8883) et tentatives de scan réseau ;
- **Contrainte** : soumise aux mêmes règles de filtrage inter-VLAN que les dispositifs IoT légitimes.

2.5 Synthèse des interactions entre composants

Les interactions suivent un cycle en cinq phases : (1) provisionnement de l'identité cryptographique via Vault ; (2) connexion MQTT/TLS 1.3 mTLS avec Mosquitto ; (3) publication sur les topics autorisés ; (4) détection par Suricata + ML Engine et corrélation Wazuh ; (5) rotation automatique des certificats. Chaque composant constitue une couche de protection indépendante : la compromission d'une couche n'invalide pas les autres.

2.6 Conclusion

Ce chapitre a présenté l'ensemble des composants du laboratoire : infrastructure réseau, broker sécurisé, PKI, simulateur de flotte, chaîne de détection et boîte à outils d'attaque. Le chapitre suivant traduit cette infrastructure en choix de conception.

Chapitre 3

Conception

3.1 Introduction

Ce chapitre traduit le diagnostic de l'existant (chapitre 1) et la connaissance des composants (chapitre 2) en une conception structurée : spécification des besoins, cas d'utilisation, vue d'ensemble de la solution, planification des sprints et modélisation UML. Chaque composant de la solution répond à une lacune identifiée au tableau 1.4.

3.2 Spécification des besoins

3.2.1 Besoins fonctionnels (BF)

Tab. 3.1 : Besoins fonctionnels

ID	Description
BF-1	Émettre, renouveler et révoquer automatiquement des certificats X.509 pour services et dispositifs.
BF-2	Imposer le chiffrement TLS 1.3 et l'authentification mutuelle (mTLS) sur le broker MQTT.
BF-3	Restreindre l'accès aux topics MQTT par dispositif via des listes de contrôle d'accès.
BF-4	Simuler une flotte hétérogène de 50 dispositifs IoT générant un trafic réaliste.
BF-5	Détecter les attaques réseau et applicatives par règles IDS et corrélérer les alertes dans un SIEM.
BF-6	Détecter les anomalies non couvertes par les règles via un modèle d'apprentissage automatique.

3.2.2 Besoins non fonctionnels (BNF)

Ces besoins définissent le *quoi* de la solution : les fonctionnalités attendues et les contraintes opérationnelles à respecter. La modélisation UML des cas d'utilisation, présentée ci-après, permet de formaliser les interactions entre les différents acteurs et la plateforme.

3.2.3 Acteurs du système

La plateforme met en jeu trois catégories d'acteurs : les **acteurs humains**, qui interagissent directement avec le système ; les **acteurs système**, qui participent au traitement de manière automatisée ; et un **acteur externe automatisé**, le dispositif IoT, qui opère de façon autonome mais constitue le point d'entrée de la chaîne de sécurité.

Tab. 3.2 : Besoins non fonctionnels

ID	Description
BNF-1	Monitoring et visualisation : tableaux de bord Wazuh/OpenSearch Dashboards pour le suivi des métriques en temps réel (latence, débit, alertes) et l'analyse de sécurité, accessibles en moins de 3 clics.
BNF-2	Performance : latence handshake mTLS < 50 ms ; débit broker > 500 msg/s.
BNF-3	Reproductibilité : l'environnement complet doit être réinstallable à partir des scripts fournis.
BNF-4	Scalabilité : l'architecture doit accepter sans refonte au moins 200 dispositifs.
BNF-5	Maintenabilité : configurations versionnées, modules indépendants, journalisation centralisée.
BNF-6	Traçabilité : chaque flux MQTT doit être identifiable par l'identité cryptographique de son émetteur ; aucun flux en clair ne doit être possible.

Acteurs système. Chaque acteur système joue un rôle spécifique dans la chaîne de sécurité :

- **PKI HashiCorp Vault** : autorité de certification automatisée, responsable de l'émission, du renouvellement et de la révocation des certificats X.509. Vault garantit que chaque dispositif et chaque service dispose d'une identité cryptographique vérifiable. En cas de compromission, il permet la révocation immédiate du certificat concerné et la mise à jour de la CRL ;
- **Broker Mosquitto** : point central de la messagerie MQTT, il assure le routage des messages entre les dispositifs et les souscripteurs. Configuré en TLS 1.3 avec mTLS obligatoire, il vérifie le certificat de chaque client lors de la connexion et applique les ACL par *Common Name* pour restreindre l'accès aux topics autorisés ;
- **IDS Suricata** : système de détection d'intrusion réseau positionné dans la zone Analyse en miroir sur le segment DMZ. Il inspecte passivement le trafic MQTT grâce à son parseur natif et génère des alertes sur les comportements suspects (flood, scan, payloads anormaux) au format `eve.json` ;
- **SIEM Wazuh** : plateforme de corrélation et de visualisation des événements de sécurité. Il agrège les alertes Suricata, les journaux Mosquitto et les scores du module ML pour fournir une vue consolidée à l'opérateur SOC via OpenSearch Dashboards ;
- **Module ML** : pipeline d'apprentissage automatique (IsolationForest + Random Forest) qui analyse les caractéristiques du trafic pour détecter les anomalies comportementales non couvertes par les règles statiques.

Acteur externe automatisé. Le **dispositif IoT** représente l'ensemble des 50 capteurs émulsés par le simulateur de flotte. Il est classé comme acteur externe car il opère de façon autonome (connexion mTLS, publication périodique, renouvellement de certificat) sans intervention humaine directe.

Acteurs humains. L'**administrateur de sécurité** est responsable de la configuration de la PKI, des ACL et des règles de détection. L'**opérateur SOC** est l'analyste chargé de la surveillance des tableaux de bord et de la réponse aux incidents.

3.2.4 Diagramme de cas d'utilisation

La figure 3.1 présente le diagramme de cas d'utilisation global de la plateforme, réunissant les trois catégories d'acteurs et les douze cas d'utilisation identifiés. Les relations d'inclusion («*include*») signalent les sous-cas obligatoires (vérification de la chaîne CA, authentification mTLS, chiffrement TLS 1.3).

Diagramme de cas d'utilisation global

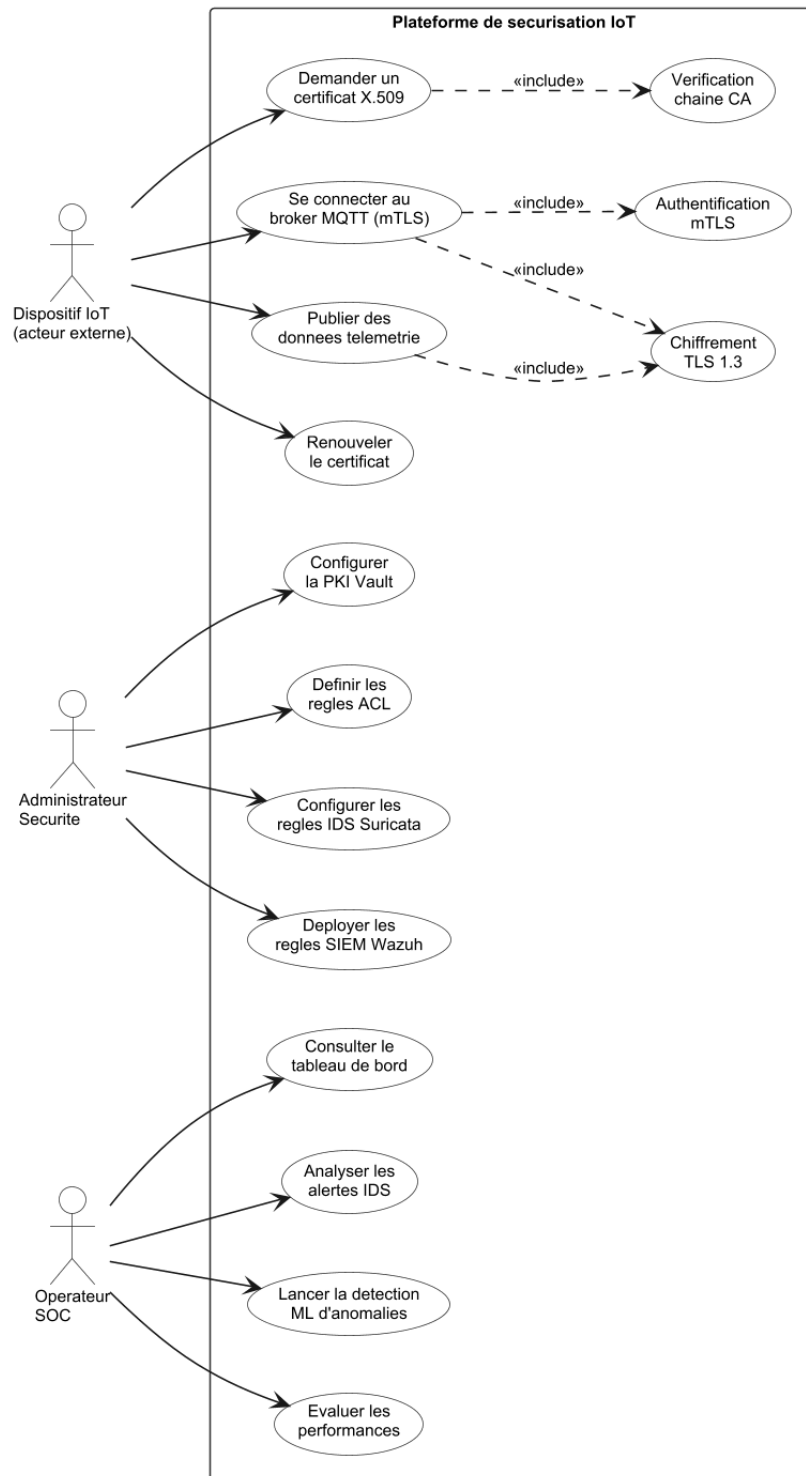


Fig. 3.1 : Diagramme de cas d'utilisation global

Le diagramme distingue trois groupes de cas d'utilisation :

- **Dispositif IoT** (acteur externe automatisé) : quatre cas couvrent le cycle de vie cryptographique et applicatif — demande de certificat X.509, connexion mTLS au broker, publication de télémétrie et renouvellement automatique du certificat. La connexion et la publication incluent obligatoirement le chiffrement TLS 1.3 ;
- **Administrateur de sécurité** : quatre cas de configuration — PKI Vault (hiérarchie CA, rôles, TTL), règles ACL Mosquitto, règles IDS Suricata et règles de corrélation SIEM Wazuh ;
- **Opérateur SOC** : quatre cas de supervision — consultation du tableau de bord Wazuh, analyse des alertes IDS, lancement de la détection ML d'anomalies et évaluation des performances de la plateforme.

Cas complémentaire : capteur erroné ou compromis. Un capteur défaillant (valeurs hors plage) ou compromis (certificat valide, données falsifiées) est détecté par la **complémentarité des mécanismes** : Mosquitto bloque les publications sur topics non autorisés (ACL), Suricata détecte les anomalies volumétriques, le module ML identifie les dérives comportementales subtiles (fréquence, contexte IP) et Wazuh corrèle ces signaux pour déclencher une alerte critique. Cette couverture multicouche justifie l'architecture en profondeur retenue.

3.3 Vue d'ensemble de la solution proposée

Pour répondre à la problématique et atteindre les objectifs ci-dessus, nous proposons une **architecture de défense en profondeur** composée de six briques complémentaires, intégralement déployée dans un laboratoire émulé sous GNS3 [22] et Docker [23] :

1. Une **infrastructure réseau segmentée** (VLANs IoT, services, monitoring, management) construite autour d'un pare-feu pfSense et d'un routeur MikroTik .
2. Une **PKI automatisée** HashiCorp Vault à deux niveaux (CA racine hors ligne + CA intermédiaire en ligne), capable d'émettre à la demande des certificats X.509 pour les services et les dispositifs [12] .
3. Un **broker MQTT Mosquitto** configuré en TLS 1.3 avec authentification mutuelle obligatoire et listes de contrôle d'accès par topic [11] .
4. Un **simulateur de flotte** de 50 dispositifs Python jouant 76 000 messages issus d'un jeu de données réel, dont sept catégories d'attaques .
5. Une **chaîne de détection** comprenant un IDS Suricata avec règles MQTT personnalisées [13] et un SIEM Wazuh [14] pour la corrélation et la visualisation .

6. Un **pipeline de ML** (IsolationForest [16] et Random Forest [17]) entraîné sur les caractéristiques extraites du trafic, capable de détecter les anomalies non couvertes par les règles statiques.

Le tableau 3.3 établit la correspondance entre les lacunes diagnostiquées à la section 1.7, les briques de la solution et les objectifs spécifiques auxquels elles répondent, assurant ainsi la traçabilité complète entre le diagnostic initial et l’architecture proposée.

Tab. 3.3 : Correspondance lacunes – briques de sécurité – objectifs spécifiques

Brique de sécurité	Section de réalisation	Obj.
Topologie réseau segmentée (GNS3, pfSense, MikroTik)	§4.3	O1
PKI Vault à deux niveaux (CA racine + CA intermédiaire)	§4.4	O2
Broker Mosquitto TLS 1.3 + mTLS + ACL par topic	§4.5	O3
Simulateur de flotte de 50 dispositifs hétérogènes	§4.6	O4
IDS Suricata avec règles MQTT dédiées	§4.6	O5
SIEM Wazuh (corrélation d’alertes et tableau de bord)	§4.6	O6
Module ML (IsolationForest + Random Forest)	§4.7	O7

La figure 3.2 schématise cette vision globale, qui sera détaillée et justifiée dans les chapitres suivants.

Le schéma fait apparaître l’empilement des six briques — segmentation réseau, PKI, mTLS, ACL, chaîne IDS/SIEM et ML — chacune couvrant une lacune identifiée au diagnostic. L’intérêt de cette représentation est de montrer que la compromission d’une couche n’invalide pas les autres, validant ainsi le principe de défense en profondeur.

La figure 3.3 détaille la topologie réseau concrète de l’architecture, en montrant le placement de chaque composant au sein des cinq zones isolées.

On y distingue la segmentation en cinq VLANs — IoT, DMZ, PKI, SIEM et Analyse — avec le broker Mosquitto en DMZ comme point de convergence des flux applicatifs, le serveur Vault isolé dans la zone PKI, Suricata en miroir sur le segment DMZ et Wazuh dans la zone SIEM. Cette disposition matérialise le principe de défense en profondeur : chaque zone ne communique qu’avec les zones strictement nécessaires selon des règles de filtrage définies sur le pare-feu pfSense.

La démarche Scrum retenue au chapitre 1 structure la réalisation en six sprints ; la section suivante en détaille la planification et les livrables.

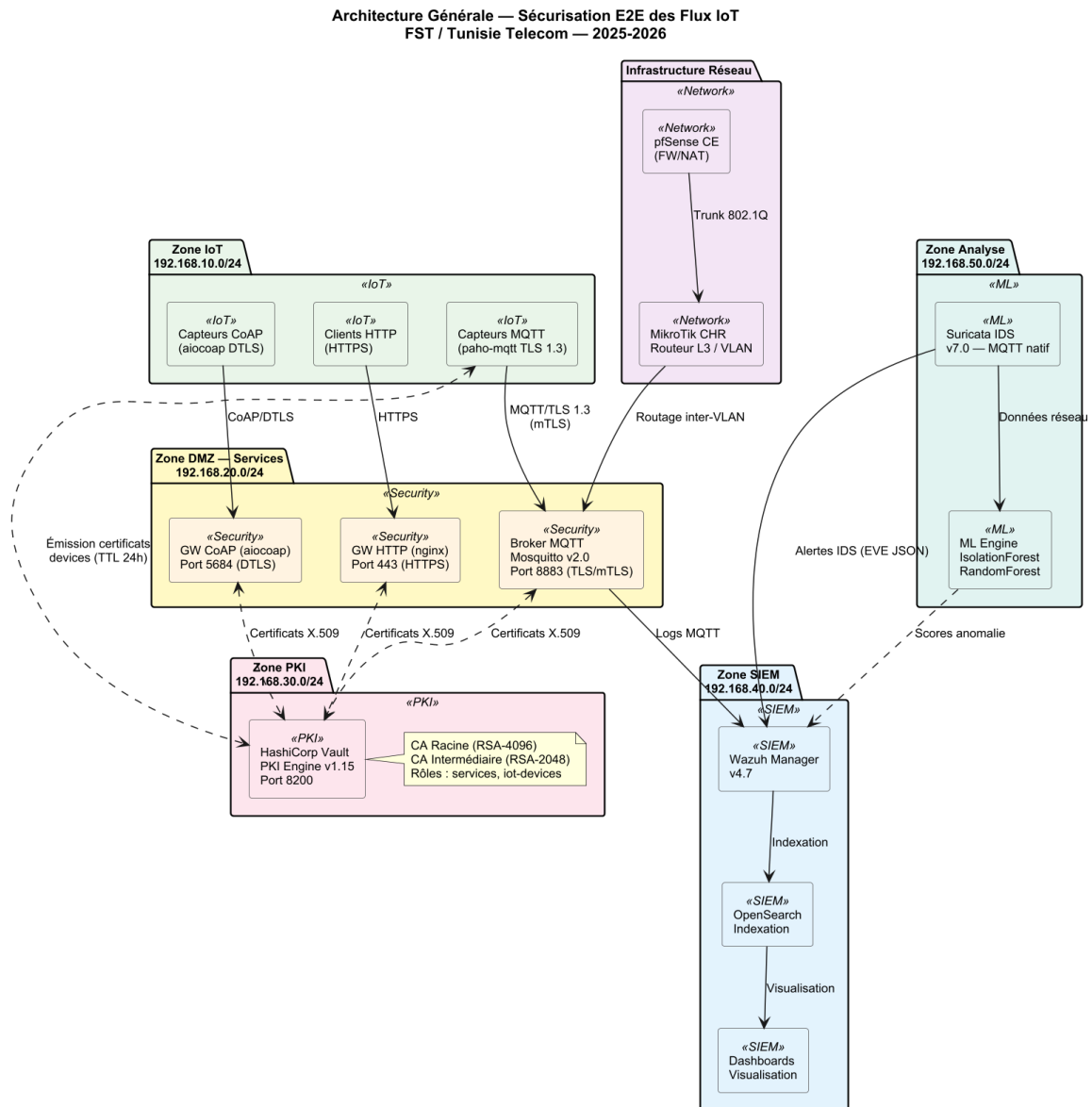


Fig. 3.2 : Vue d'ensemble de l'architecture

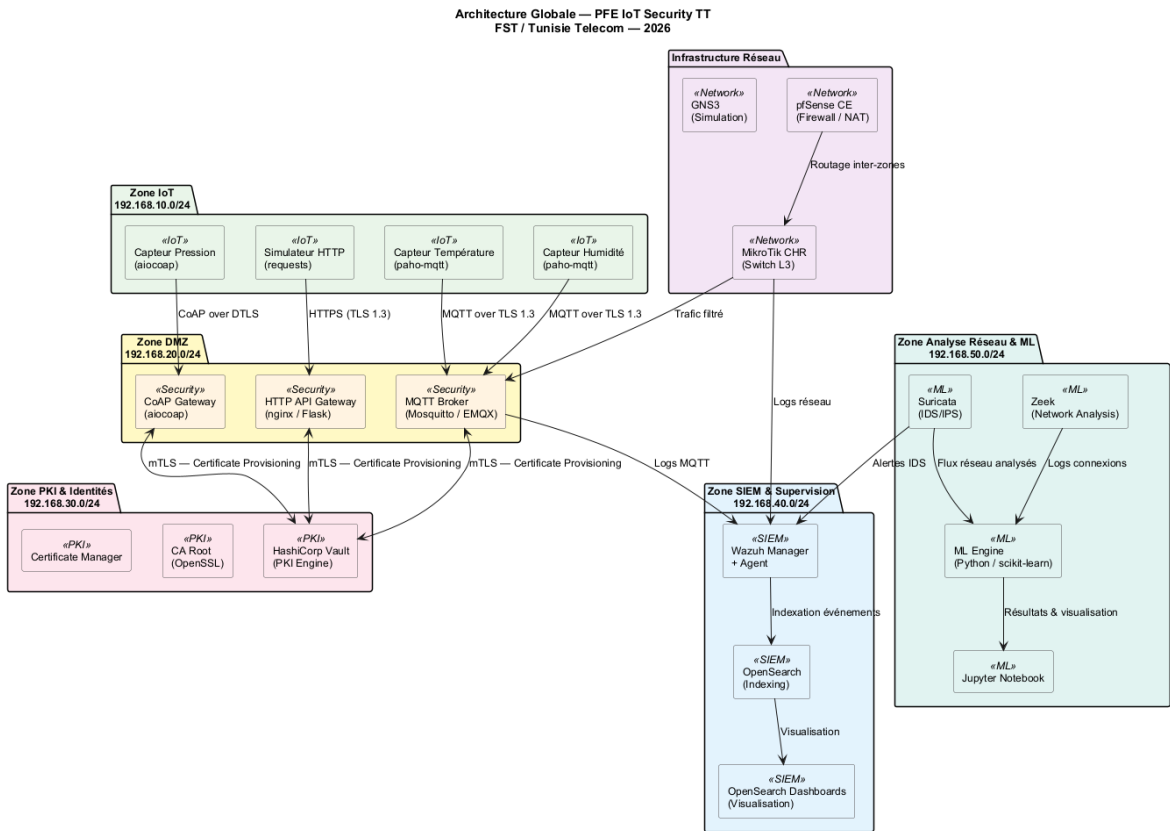


Fig. 3.3 : Architecture générale du système

3.4 Planification des sprints

Six sprints de deux semaines structurent la phase de réalisation. Le tableau 3.4 en présente les objectifs principaux, du cadrage réseau initial jusqu'à l'évaluation du module ML.

Tab. 3.4 : Planification des sprints

Sprint	Durée	Objectif principal
S1	2 semaines	Cadrage réseau, topologie segmentée, conteneurs de base et plan d'adressage.
S2	2 semaines	Déploiement de Vault, CA racine et intermédiaire, rôles PKI, scripts d'émission.
S3	2 semaines	Configuration de Mosquitto en TLS 1.3+mTLS, ACL par certificat et tests d'authentification.
S4	2 semaines	Simulation de 50 clients Python, génération de trafic et rejeu de 76 000 messages.
S5	2 semaines	Règles Suricata MQTT, intégration Wazuh et tableaux de bord.
S6	2 semaines	Modèles IsolationForest et RandomForest, évaluation finale et rédaction.

La figure 3.4 présente cette planification sous forme de diagramme de Gantt, illustrant la dépendance séquentielle entre les sprints et les jalons clés du projet.

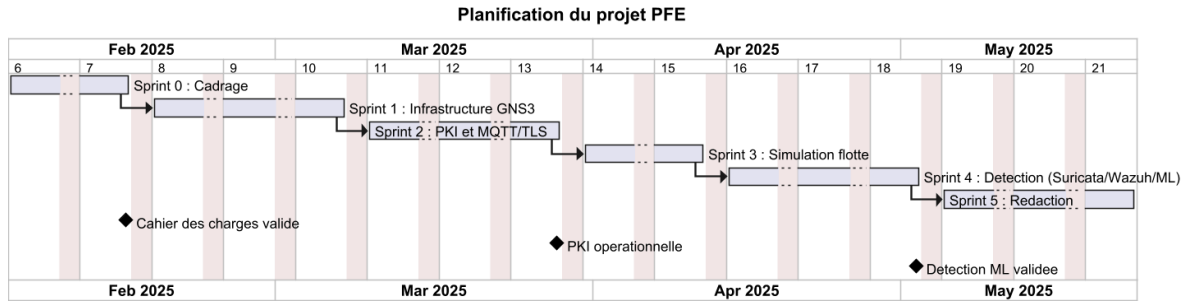


Fig. 3.4 : Diagramme de Gantt

Cette planification met en évidence une progression cohérente : l'infrastructure et la PKI précèdent logiquement le broker sécurisé, puis la simulation, la détection et enfin l'évaluation ML.

3.5 Backlog produit

Le *backlog* a été priorisé selon **MoSCoW** et réparti sur les six sprints. Les éléments critiques couvrent la topologie, la PKI, le broker mTLS, la simulation, l'intégration Suricata/Wazuh et le module ML.

3.6 Modélisation UML

La modélisation UML a été recentrée sur trois vues suffisantes pour comprendre la solution : la structure du simulateur, l'émission d'un certificat client et le cycle de publication mTLS.

3.6.1 Diagramme de classes du simulateur

La figure 3.5 résume le noyau d'orchestration du simulateur : **FleetOrchestrator** pilote les instances **DeviceSimulator**, tandis que **DeviceStats** centralise les métriques utiles à l'évaluation.

Diagramme de Classes --- Partie 1 : Orchestration de la simulation

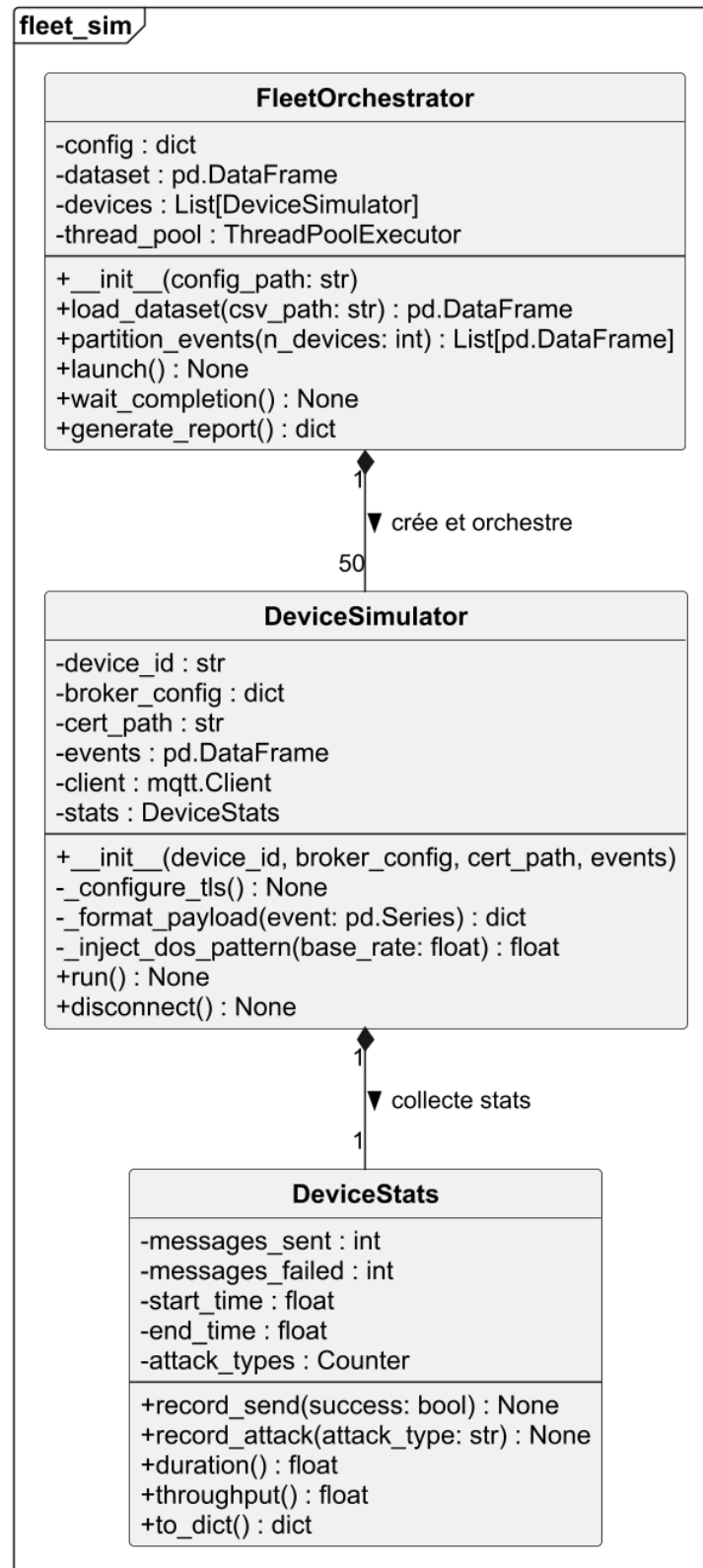


Fig. 3.5 : Diagramme de classes du simulateur

3.6.2 Diagramme de séquence : émission d'un certificat client

La figure 3.6 résume l'émission d'un certificat client via Vault : requête API authentifiée, contrôle du rôle PKI, signature par la CA intermédiaire puis retour du bundle au dispositif, qui peut ensuite programmer son renouvellement.

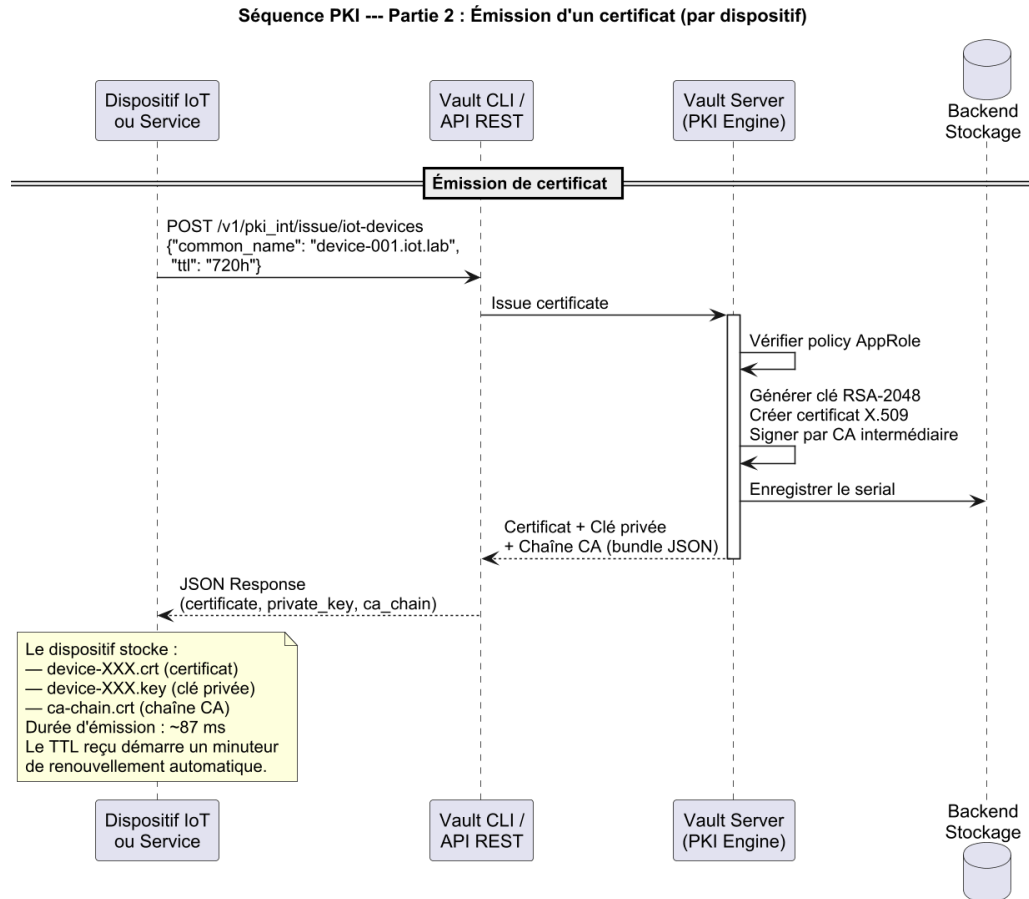


Fig. 3.6 : Séquence d'émission d'un certificat client

3.6.3 Diagramme d'activité : cycle de publication mTLS

La figure 3.7 synthétise le comportement d'un dispositif simulé : chargement de son sous-ensemble de données, publication MQTT sur les topics autorisés, adaptation du rythme d'émission en cas d'attaque, puis agrégation des statistiques en fin d'exécution.

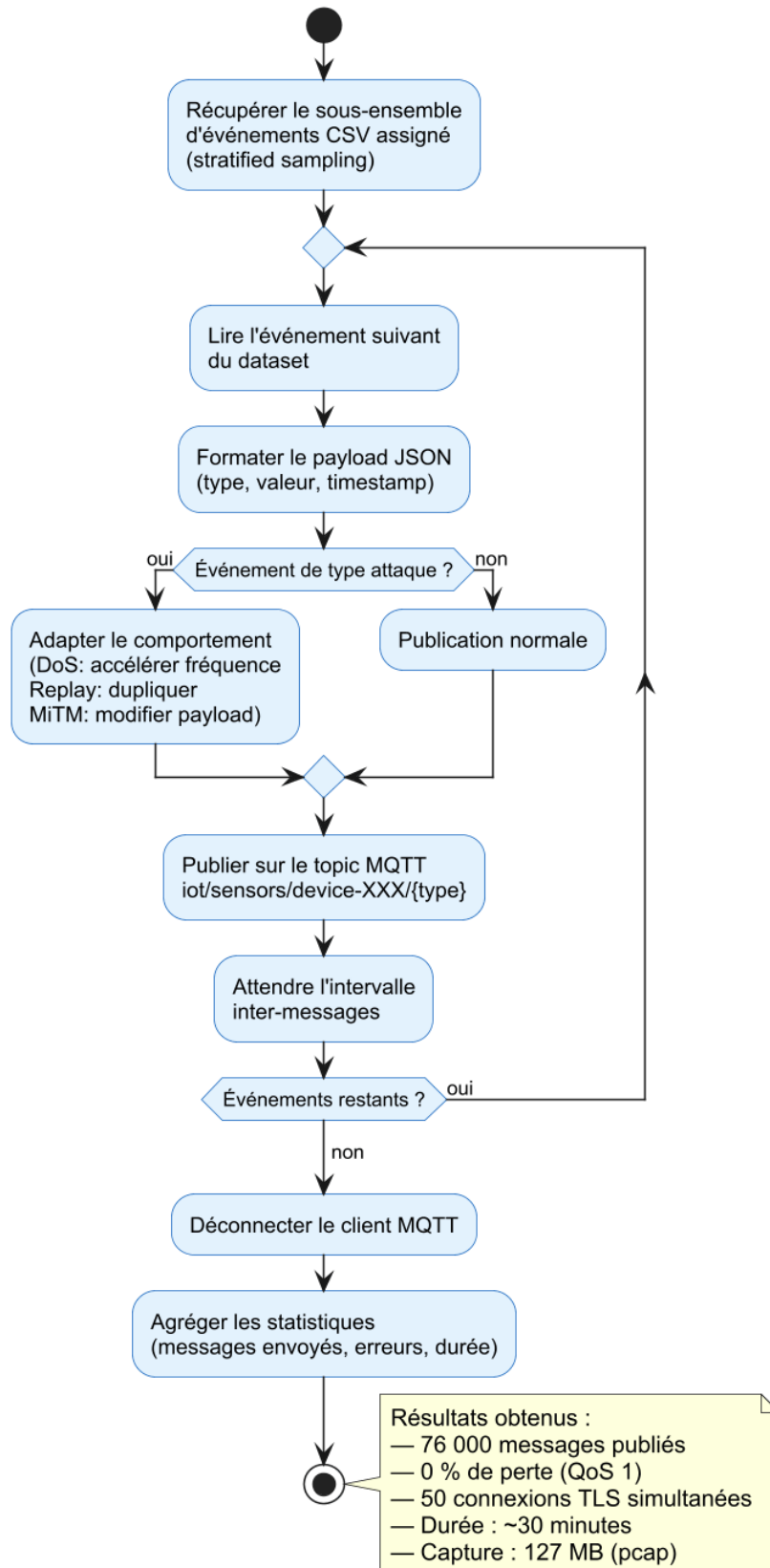
Activité --- Partie 3 : Boucle de publication MQTT

Fig. 3.7 : Cycle de publication mTLS

3.6.4 Diagrammes d'activité des scénarios d'attaque

Pour compléter la modélisation, deux diagrammes d'activité illustrent les scénarios d'attaque les plus représentatifs de l'architecture de détection multicouche. Le premier (figure 3.8) décrit un scénario de déni de service par flood CONNECT, détecté au niveau réseau. Le second (figure 3.9) décrit un scénario de compromission de certificat, détecté par le module ML.

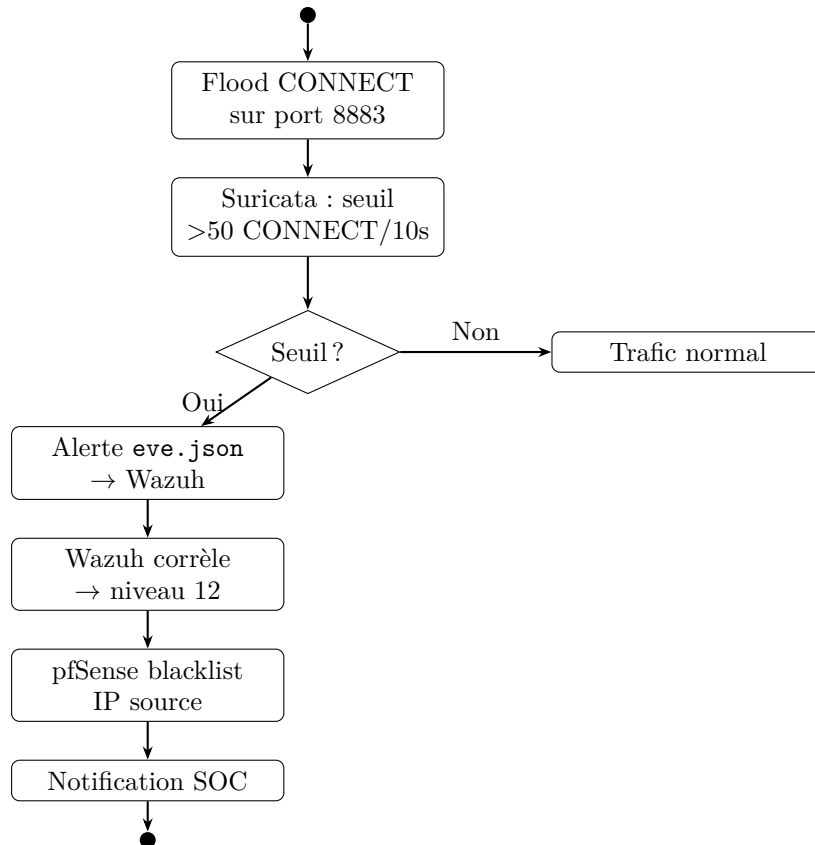


Fig. 3.8 : Scénario S5 : flood CONNECT

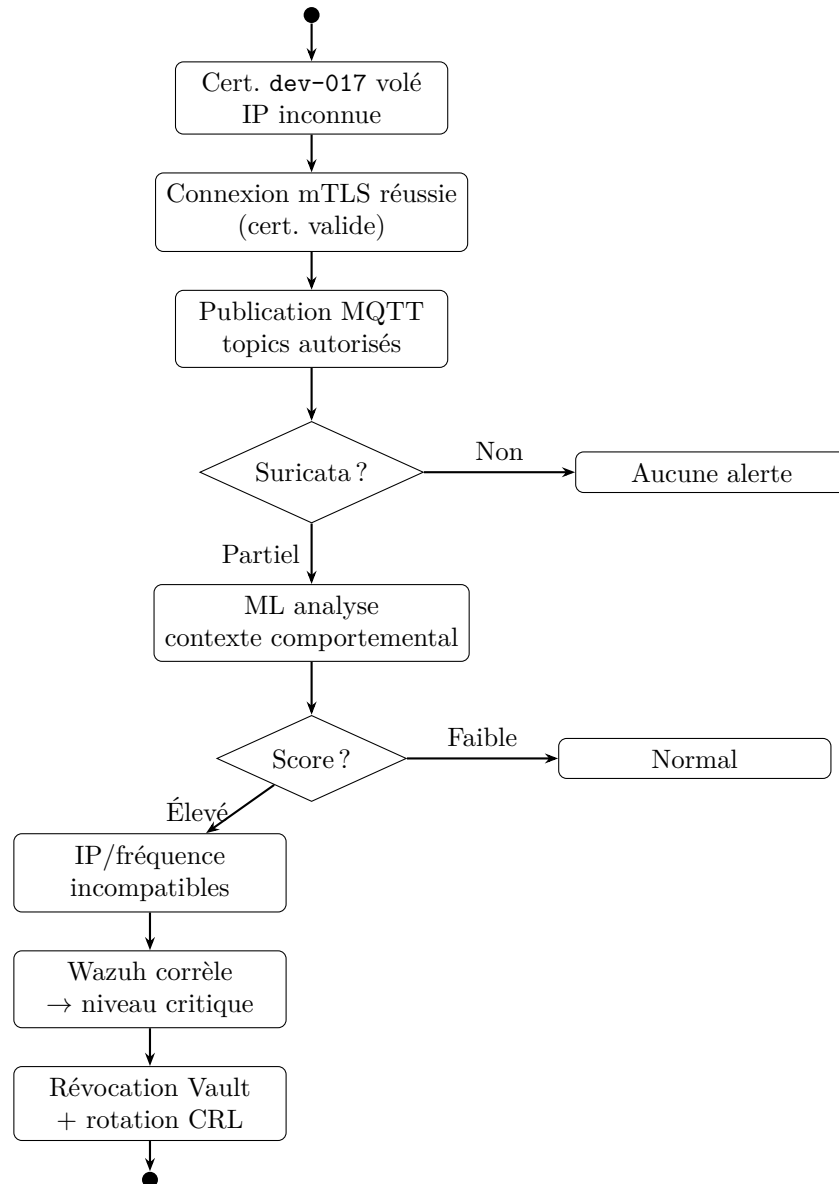


Fig. 3.9 : Scénario S6 : compromission de certificat

Le scénario S5 illustre la détection au **niveau réseau** : Suricata identifie le flood, Wazuh corrèle les alertes et pfSense bloque dynamiquement la source. Le scénario S6 illustre la détection au **niveau comportemental** : le certificat étant techniquement valide, seul le module ML est capable de détecter l'écart par rapport au profil historique du dispositif. Ces deux diagrammes démontrent la complémentarité des couches de défense, justifiant l'architecture en profondeur retenue.

3.7 Conclusion

Ce chapitre a formalisé la conception : besoins, acteurs, cas d'utilisation, architecture de défense en profondeur, planification et modélisation UML. Le chapitre suivant décrit la mise en œuvre concrète de chaque brique.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la mise en œuvre concrète de l’architecture : topologie réseau, PKI Vault, broker MQTT mTLS, scénarios d’attaque, et pipeline ML.

4.2 Environnement de développement

Le laboratoire est entièrement reproductible sur une station Windows 10/11 (16 Go de RAM minimum). L’inventaire complet des composants, de leurs versions et de leurs adresses réseau est présenté au chapitre 2 (tableau 2.1). Ces composants ne fonctionnent pas de manière indépendante : ils s’articulent en une **architecture de défense en profondeur** où chaque couche renforce les autres. La section suivante décrit cette organisation globale et le flux de données de bout en bout, avant d’entrer dans le détail de chaque brique.

4.3 Architecture globale et topologie réseau

4.3.1 Topologie réseau

La figure 4.1 présente la topologie réseau du laboratoire : la chaîne Internet → pfSense → MikroTik et la segmentation en cinq VLANs, avec un miroir passif du trafic DMZ vers Suricata (zone Analyse). L’architecture générale du système a déjà été présentée au chapitre 3 (figure 3.3) et l’inventaire complet des composants figure au chapitre 2 ; la présente section en détaille la topologie réseau effective.

**Topologie réseau --- Partie 1 : Infrastructure réseau
pfSense, MikroTik et segmentation VLAN (vue globale)**

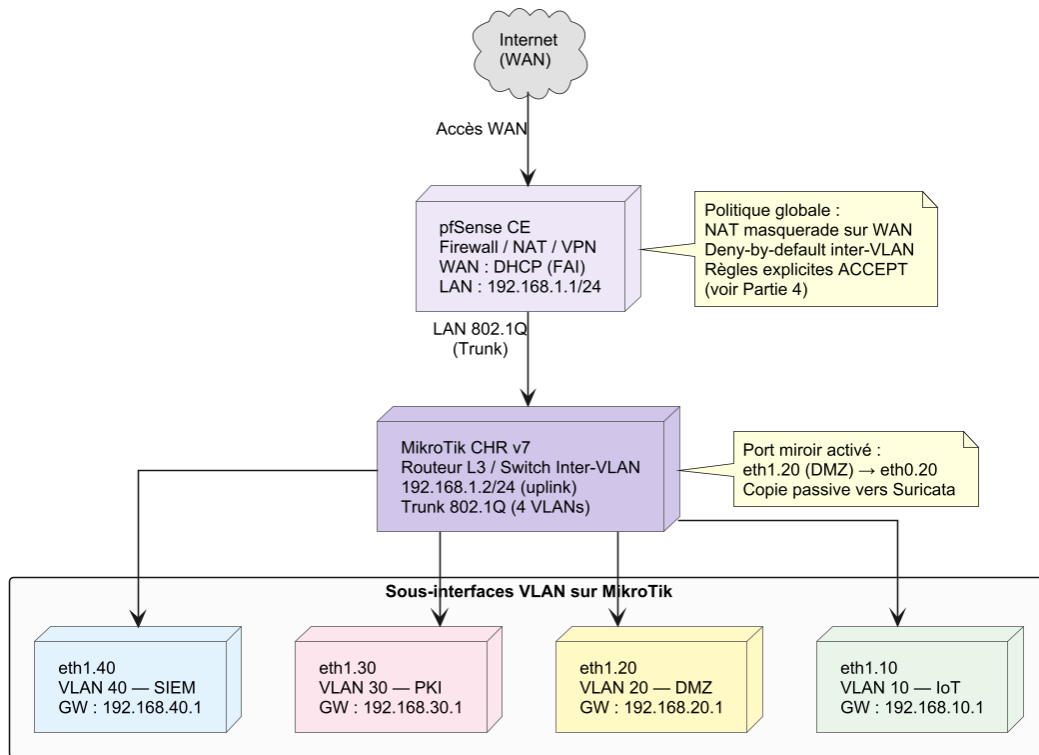


Fig. 4.1 : Topologie réseau

La segmentation repose sur cinq zones isolées, conformément à l'inventaire détaillé au chapitre 2 :

- **VLAN 10 (Zone IoT)** : héberge le simulateur de flotte et les 50 dispositifs émulés. Les seuls flux sortants autorisés sont IoT→Broker/8883 (MQTT/TLS) et IoT→Vault/8200 (émission de certificats) ;
- **VLAN 20 (Zone DMZ)** : contient exclusivement Mosquitto (broker MQTT). Un port miroir MikroTik copie passivement le trafic de cette zone vers Suricata dans la zone Analyse ;
- **VLAN 30 (Zone PKI)** : isole HashiCorp Vault et la CA intermédiaire ;
- **VLAN 40 (Zone SIEM)** : regroupe Wazuh Manager, OpenSearch et OpenSearch Dashboards. Les journaux de toutes les zones convergent ici via Syslog/1514 ;
- **VLAN 50 (Zone Analyse)** : héberge Suricata IDS (192.168.50.10) et le ML Engine (192.168.50.12). Suricata reçoit le trafic miroir de la DMZ ; le ML Engine se connecte au broker en tant que client mTLS. Les alertes sont transmises à Wazuh (SIEM) via TCP/1514.

La politique de filtrage inter-VLAN suit le principe *deny-by-default* et repose sur un **double barrage** : le **MikroTik** applique les règles de filtrage inter-VLAN dans

sa chaîne *forward* entre les cinq bridges, tandis que **pfSense** renforce cette politique par des règles complémentaires sur son interface LAN, formant ainsi deux couches de filtrage indépendantes.

Le cycle de fonctionnement de bout en bout suit cinq phases successives : (1) provisionnement de l'identité cryptographique via Vault (zone PKI) ; (2) établissement de la connexion TLS 1.3 mTLS avec le broker (zone DMZ) ; (3) publication des données IoT sur les topics autorisés ; (4) détection par Suricata (zone Analyse) et corrélation par Wazuh (zone SIEM) ; (5) rotation automatique des certificats à l'approche de leur expiration. Les deux phases les plus critiques — l'établissement du mTLS et la détection — sont détaillées ci-après. La figure 4.2 illustre la phase 2.

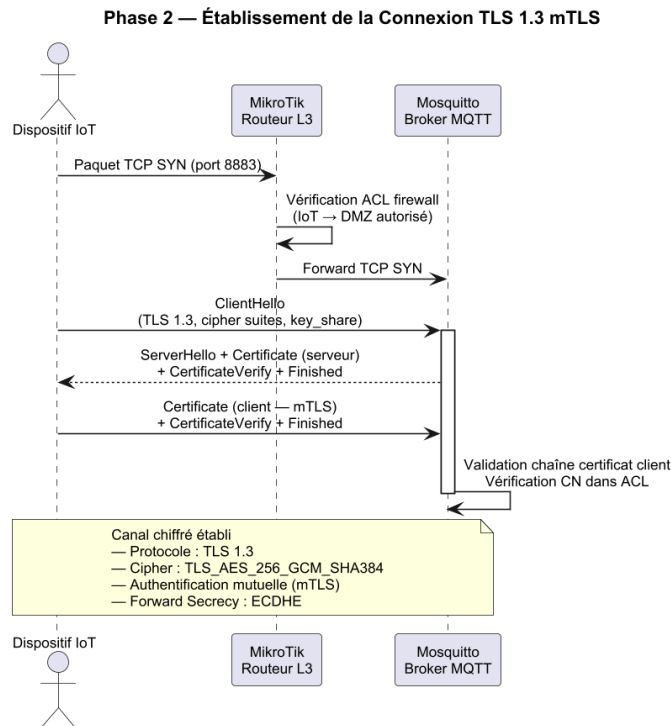


Fig. 4.2 : Flux E2E — Phase 2 : connexion mTLS

Le diagramme montre que le dispositif présente son certificat client, que le broker vérifie la chaîne PKI complète, et que les deux parties dérivent les clés de session avant tout échange applicatif. Ce mécanisme garantit qu'aucun flux MQTT ne circule sans preuve mutuelle d'identité. La figure 4.3 illustre la phase 4.

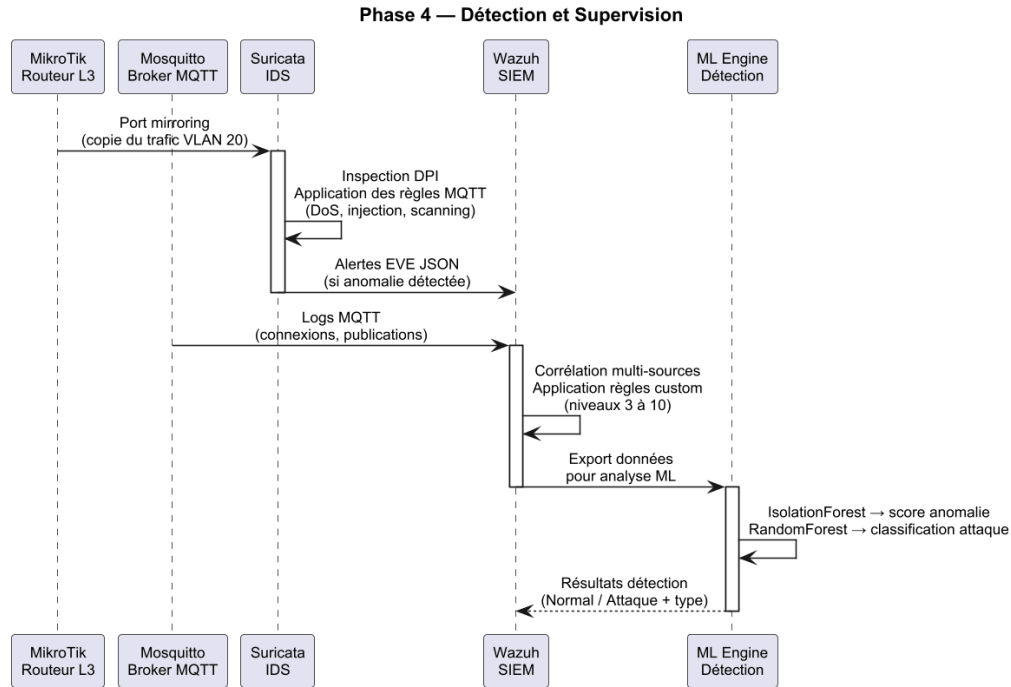


Fig. 4.3 : Flux E2E — Phase 4 : détection et supervision

Cette figure révèle le chemin complet d’une alerte : Suricata inspecte le trafic miroir, produit un événement `eve.json`, que Wazuh corrèle avec les journaux Mosquitto avant de le restituer sur le tableau de bord SOC. La chaîne complète ainsi la boucle de sécurité en ajoutant la détection aux mécanismes préventifs.

4.4 Infrastructure à clés publiques (Vault)

4.4.1 Hiérarchie

La PKI suit le modèle à deux niveaux préconisé par le NIST [7] : une CA racine *offline* (générée hors ligne, conservée chiffrée) signe une unique CA intermédiaire en ligne, gérée par Vault [12]. La figure 4.4 illustre cette hiérarchie.

Le diagramme distingue les trois niveaux de confiance de la hiérarchie : (i) la **CA racine** (RSA-4096, self-signed, TTL 10 ans) est conservée hors ligne et ne signe que les CA intermédiaires ; sa clé privée n’est jamais exposée dans la VM GNS3 ; (ii) la **CA intermédiaire** (RSA-4096, TTL 5 ans) est embarquée dans le *secrets engine PKI* de Vault et signe les certificats opérationnels à la demande via API REST ; (iii) les **certificats finaux** sont émis selon deux rôles distincts : **iot-services** (TTL par défaut 30 jours, max 1 an, flags `serverAuth+clientAuth`, pour Mosquitto, Suricata et le ML Engine), et **iot-devices** (TTL par défaut 24 heures, max 30 jours, flag `clientAuth` uniquement, CN contrôlé au pattern `dev-XXX.iot.lab`). La séparation des rôles empêche qu’un certificat de service soit utilisé pour s’authentifier comme dispositif, et inversement.

**Diagramme de Classes — Hiérarchie PKI
Infrastructure à Clés Publiques**

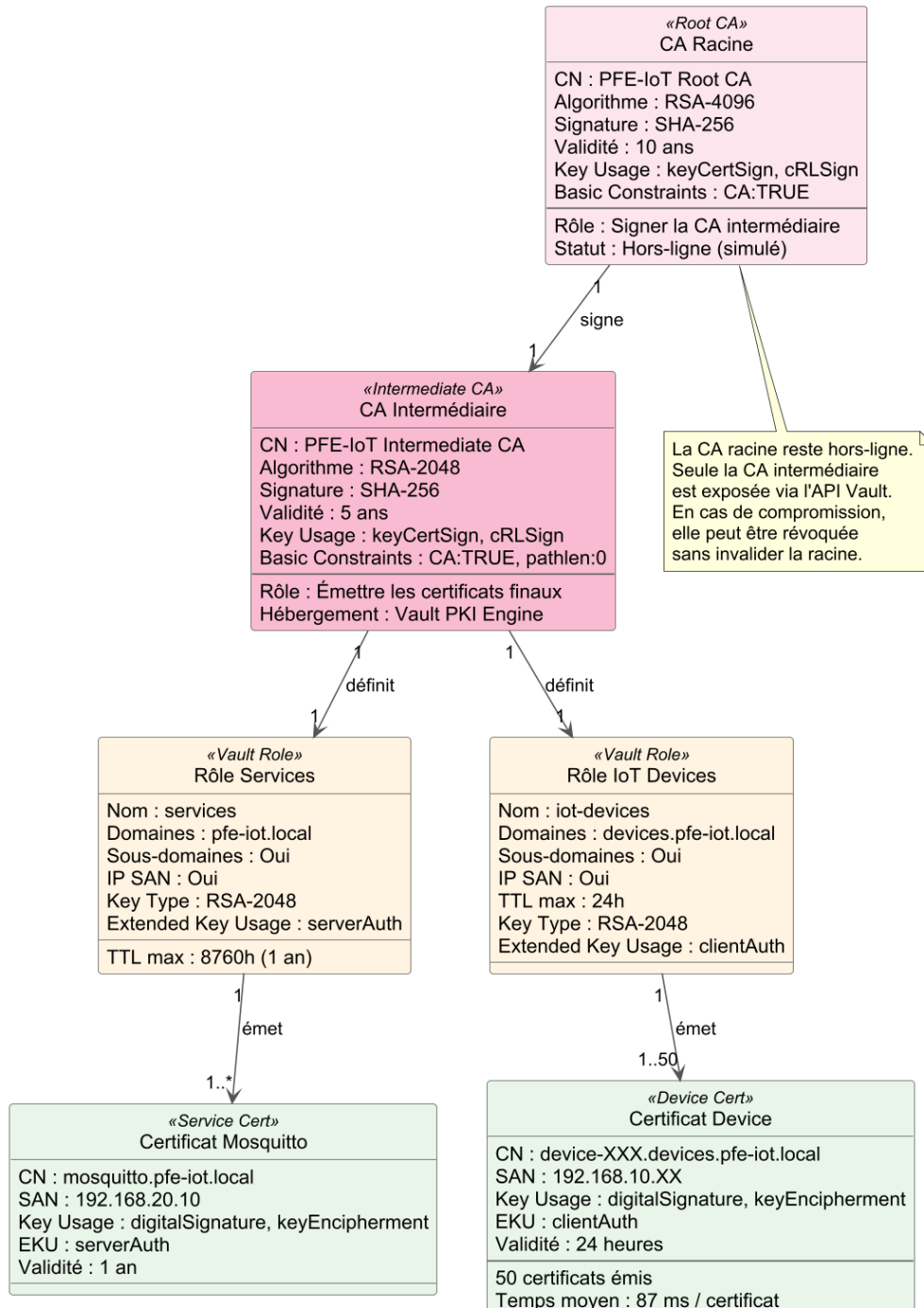


Fig. 4.4 : Hiérarchie PKI à deux niveaux

4.4.2 Rôles Vault et émission

Les deux rôles décrits ci-dessus sont définis dans le script `bootstrap-pki-api.sh`. L'émission d'un certificat client se réduit à un appel HTTP authentifié par jeton `AppRole`.

```
1 vault write -format=json pki_int/issue/iot-devices \
2     common_name="dev-042.iot.lab" \
3     ttl="720h"
```

Listing 4.1: Émission d'un certificat client via Vault

Le choix des TTL résulte du compromis **sécurité / coût opérationnel** : un TTL court réduit la fenêtre d'exposition mais impose une rotation fréquente. Pour les services d'infrastructure, 30 jours (max 1 an) équilibre stabilité et rotation ; pour les dispositifs, 24 heures (max 30 jours) est préférable car la rotation est automatisée par Vault, conformément aux recommandations du NIST [7].

La PKI étant opérationnelle, le broker MQTT peut désormais exiger un certificat signé par la CA intermédiaire avant d'accepter toute connexion.

4.5 Broker MQTT en TLS 1.3 + mTLS

Mosquitto est configuré [11] pour exiger un certificat client, associer l'identité certifiée au nom d'utilisateur et appliquer un fichier d'ACL liant chaque *Common Name* à ses topics autorisés. La figure 4.5 retrace le *handshake* TLS 1.3 mutuel observé entre un dispositif et le broker.

Le diagramme de séquence se lit selon quatre phases distinctes :

1. **ClientHello** (dispositif → broker) : le dispositif annonce les suites cryptographiques qu'il supporte et propose les paramètres de partage de clé (ECDHE) ;
2. **ServerHello + CertificateVerify** (broker → dispositif) : le broker sélectionne la suite, transmet son certificat X.509 signé par la CA Vault et prouve la possession de sa clé privée ;
3. **Certificate (client)** (dispositif → broker) : le dispositif présente son propre certificat (rôle `iot-devices`, CN `dev-XXX.iot.lab`) ; le broker vérifie sa validité, son appartenance à la chaîne PKI et l'absence de révocation dans la CRL Vault ;
4. **Finished + Application Data** : les deux parties dérivent les clés de session symétriques (HKDF sur secret ECDHE) et la session chiffrée est établie en **1-RTT**.

L'authentification mutuelle garantit qu'aucun dispositif sans certificat valide ne peut accéder au broker, et qu'aucun broker frauduleux ne peut se faire passer pour Mosquitto.

Pour valider la robustesse de cet ensemble, six scénarios d'attaque ont été conçus.

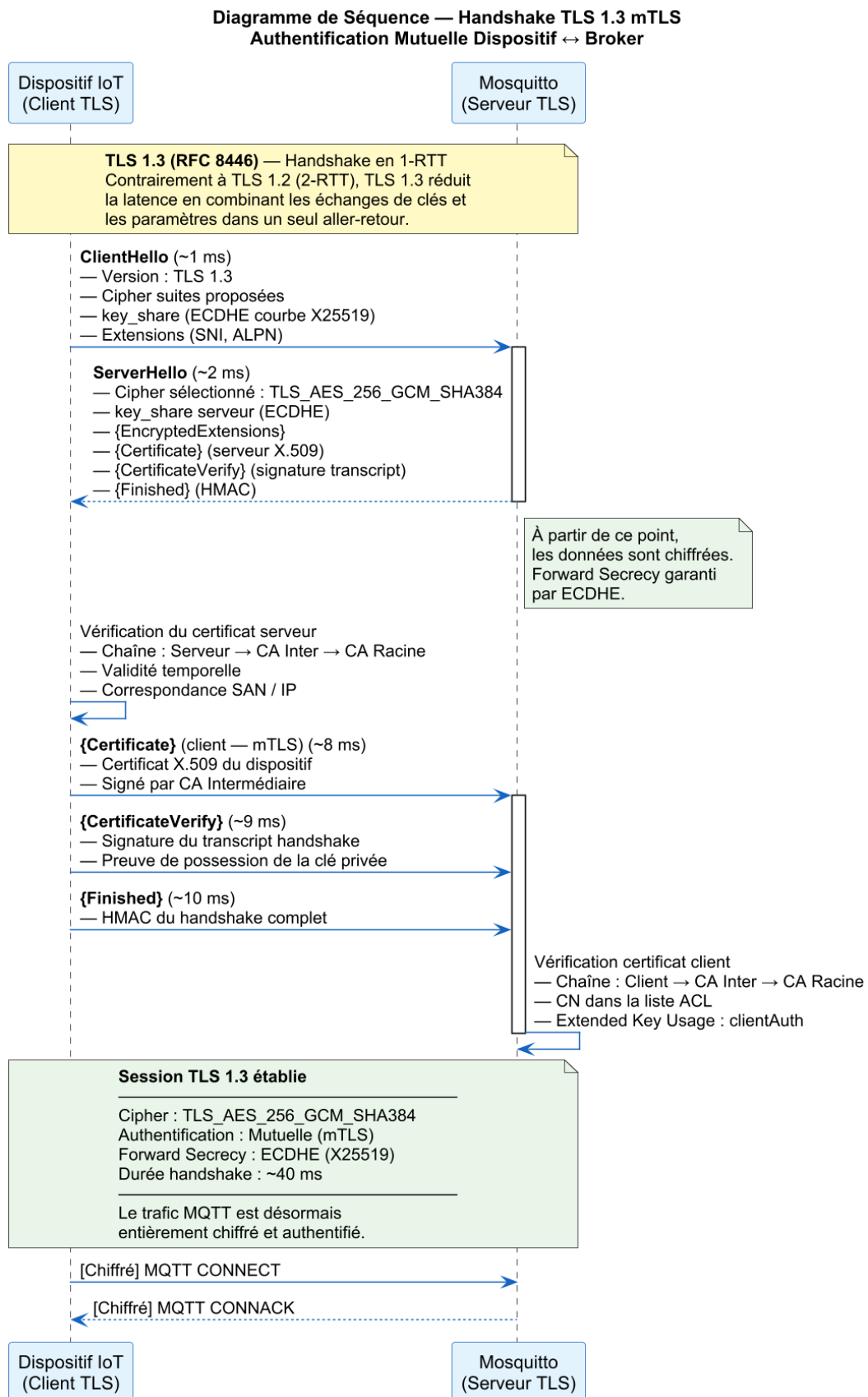


Fig. 4.5 : Handshake TLS 1.3 mTLS

4.6 Scénarios d'attaque et contre-mesures

Six scénarios couvrent les principales catégories d'attaques applicables à un broker MQTT. Le tableau 4.1 les synthétise avec leur vecteur, leur méthode de détection et la contre-mesure appliquée.

Tab. 4.1 : Scénarios d'attaque, détection et contre-mesures

ID	Vecteur	Détection	Contre-mesure
S1 – MITM	Connexion port 1883 (clair) ; MITM capture/modifie les messages	Règle Suricata TCP/1883	pfSense bloque 1883 ; broker sur 8883 + require_certificate
S2 – Sniffing	Interception passive MQTT non chiffré	Alerte Suricata hors session TLS	TLS 1.3 obligatoire (RFC 9325 [10])
S3 – Brute-force	500 CONNECT/60s avec certificats forgés	Seuil CONNECT/IP Suricata	Certificats CA Vault ; <i>rate-limit</i> pfSense
S4 – Exfiltration	SUBSCRIBE à tt/admin/#	Log ACL denied + corrélation Wazuh	ACL stricte par CN, granularité topic
S5 – Flood	Milliers de connexions TLS partielles	Taux TLS anormal Suricata	max_connections ; blacklist Wazuh
S6 – Cert. volé	Certificat dev-017 réutilisé depuis IP inconnue	IF (écart comportemental) + corrélation Wazuh	Révocation Vault + rotation CRL

La chaîne de détection fonctionne en trois étapes : Suricata reçoit le trafic VLAN 20 via port miroir MikroTik (AF_PACKET, zéro copie), active son parseur MQTT sur le port 8883, puis applique les règles SID personnalisées. La figure 4.6 illustre cette chaîne.

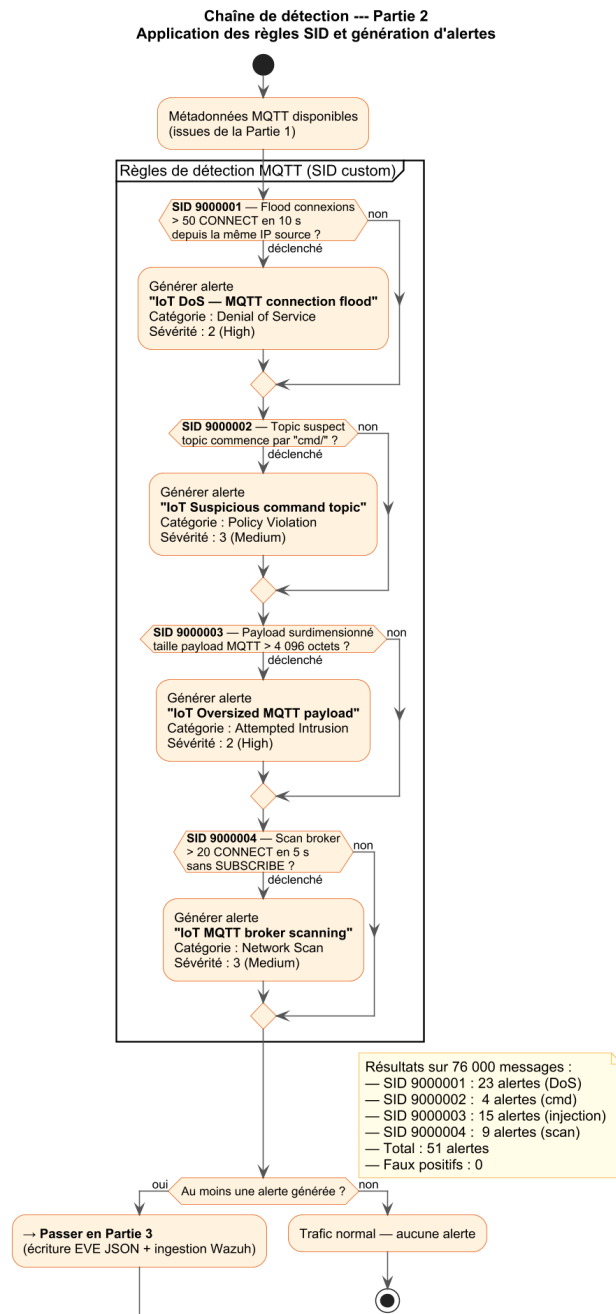


Fig. 4.6 : Chaîne de détection Suricata–Wazuh

L'analyse de cette figure révèle quatre règles SID personnalisées : **SID 9000001** (flood : > 50 CONNECT en 10 s depuis la même IP, 23 déclenchements) ; **SID 9000002** (topic suspect `cmd/`, 4 déclenchements) ; **SID 9000003** (payload > 4 096 octets, 15 déclenchements) ; **SID 9000004** (scan broker : > 20 CONNECT en 5 s sans SUBSCRIBE, 9 déclenchements). Le total de 51 alertes sur 76 000 messages, avec 0 faux positif sur le trafic TLS normal, démontre la précision du parseur MQTT natif. La figure 4.7 détaille le pipeline d'ingestion qui prend le relais en aval.

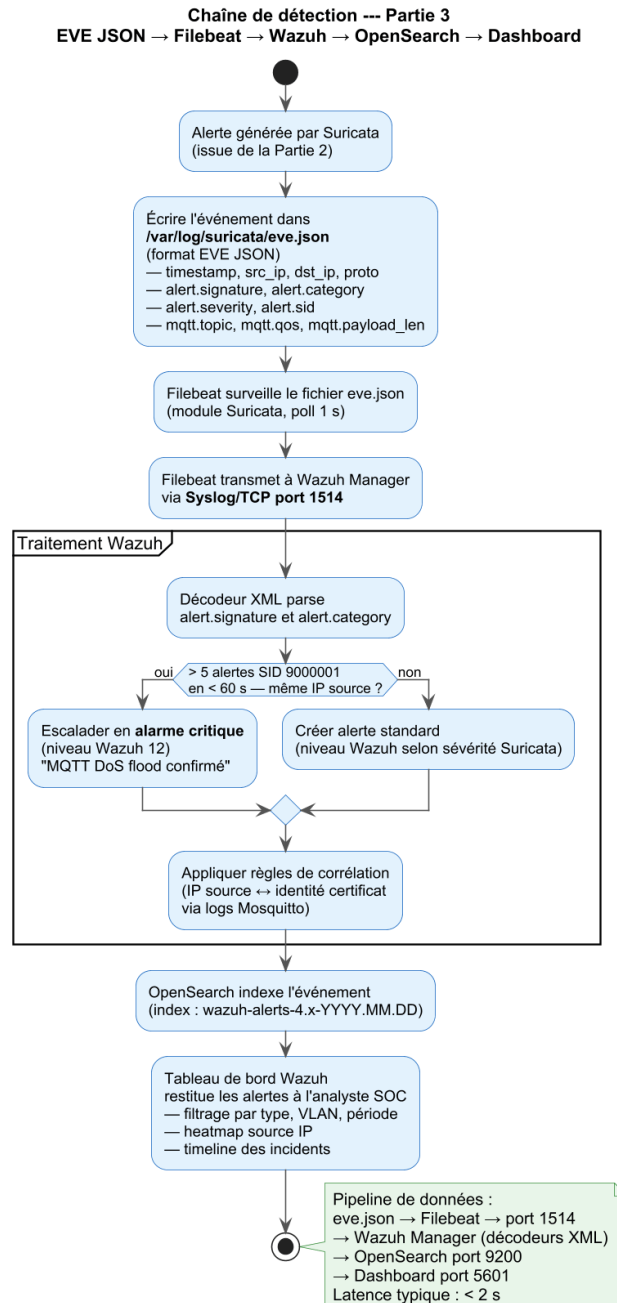


Fig. 4.7 : Pipeline d'ingestion des alertes

Le pipeline fonctionne ainsi : Suricata écrit chaque alerte dans `eve.json` (champs : `timestamp`, `src_ip`, `alert.signature`, `mqtt.topic`), Filebeat la transmet à Wazuh

via Syslog/1514. Wazuh applique ses décodeurs XML puis ses règles de corrélation (ex. : > 5 alertes SID 9000001 en < 60 s $\rightarrow$ escalade niveau 12 « MQTT DoS flood confirmé »). OpenSearch indexe les événements et le tableau de bord les restitue à l'analyste SOC avec filtrage par type d'attaque, VLAN source et période. La latence bout-en-bout inférieure à 2 secondes confirme la viabilité opérationnelle de cette chaîne.

4.7 Pipeline d'apprentissage automatique

Le scénario S6 (certificat valide réutilisé) n'est que partiellement détecté par les règles. Le pipeline ML comble cette lacune en trois phases :

Phase 1 — Chargement, exploration et feature engineering. Le dataset CSV (76 000 entrées) est chargé puis soumis à une analyse exploratoire : distribution des classes, matrice de corrélation et boxplots par type d'attaque. Il comprend **29 features** réparties en deux familles :

- **21 features numériques** : caractéristiques volumétriques (`payload_bytes`, `packet_count`, `session_duration_ms`), temporelles (`rtt_ms`, `jitter_ms`, `topic_frequency_1m`), de sécurité (`failed_auth_1h`, `conn_attempts_5m`, `key_rotation_age_days`), comportementales (`payload_entropy` calculée par entropie de Shannon, `duplicate_msg_rate_pct`, `anomaly_score_baseline`, `risk_score`) et environnementales (`cpu_usage_pct`, `mem_usage_pct`, `signal_rssi_dbm`, `battery_pct`);
- **8 features catégorielles** : `protocol`, `tls_version`, `auth_method`, `cert_status`, `src_zone`, `dst_zone`, `message_type` et `device_type`, encodées via `LabelEncoder` pour produire 29 features numériques exploitables.

Deux variables cibles sont construites : `label_secure` (binaire : 0 = attaque, 1 = normal) et `attack_type` (multi-classe : 7 types d'attaques). L'ordre des features et le dictionnaire des encodeurs sont sauvegardés dans `ml/models/` afin de garantir la cohérence entre l'entraînement et l'inférence en ligne.

Phase 2 — Entraînement des trois modèles. Les trois modèles sont entraînés en parallèle sur les mêmes 29 features numériques (figure 4.8).

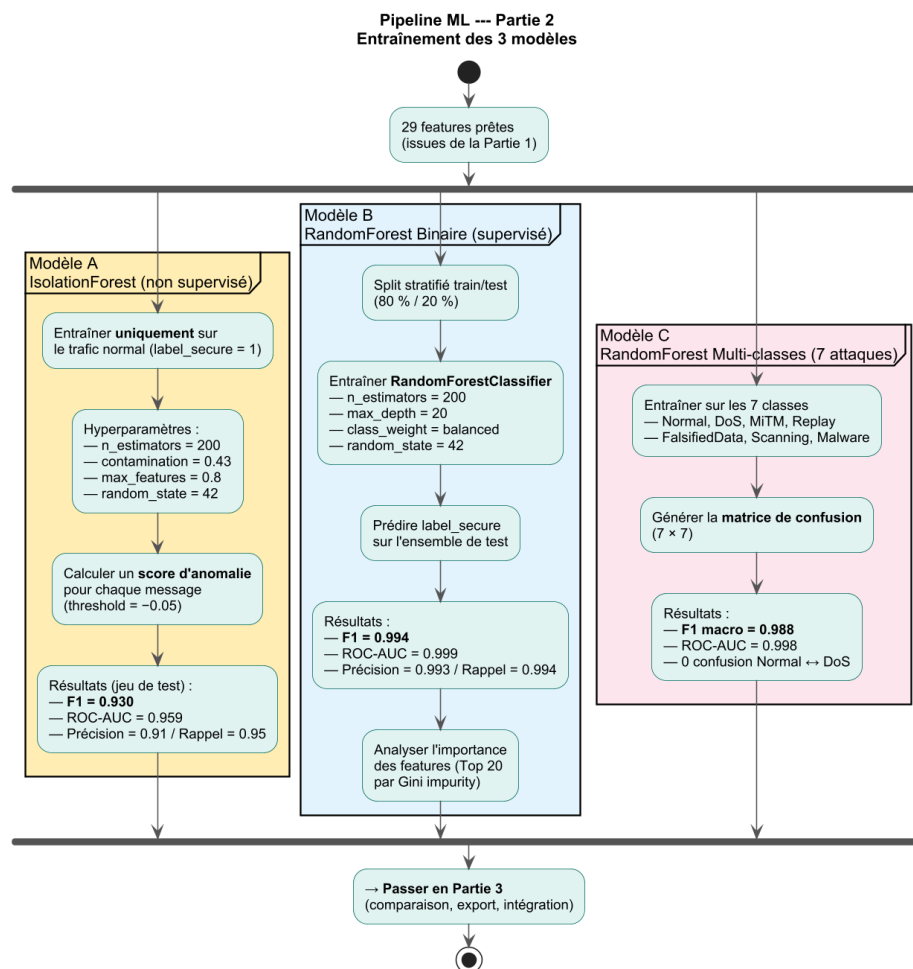


Fig. 4.8 : Pipeline ML

- **Modèle A — IsolationForest** (non supervisé) : entraîné exclusivement sur le trafic normal (`label_secure = 1`), il calcule un score d'anomalie pour chaque message, avec un seuil fixé à $-0,05$. Résultats : $F1 = 0,932$ et $ROC-AUC = 0,959$;
- **Modèle B — RandomForest binaire** (supervisé) : entraîné avec un split stratifié 80/20, `class_weight=balanced` et `n_estimators=200`. Résultats : $F1 = 0,994$ et $ROC-AUC = 0,999$;
- **Modèle C — RandomForest multi-classes** : classification directe des 7 types d'attaques, matrice de confusion sans confusion Normal $\leftrightarrow$ DoS. Résultats : $F1$ macro = $0,993$ et $ROC-AUC = 0,998$.

Phase 3 — Comparaison, export et intégration SIEM. Les trois modèles sont comparés sur $F1$, ROC-AUC, précision et rappel. La stratégie retenue est bicouche : IsolationForest pour la détection sans étiquettes (données de production non labellisées) et RandomForest binaire pour la classification en temps réel du trafic labellé. Les modèles sont sauvegardés au format `.pkl` (joblib) dans `ml/models/`. Les scores d'anomalie sont transmis à Wazuh via l'API REST (port 55000) et corrélés avec les alertes Suricata pour une détection multicouche : un message simultanément signalé par l'IDS et classé anormal par le modèle ML déclenche une alarme de niveau critique.

4.7.1 Données et features

Le jeu de données utilisé contient 76 000 enregistrements répartis en huit classes (1 normale + 7 attaques) [15]. Les figures 4.9 et 4.10 présentent l'analyse exploratoire.

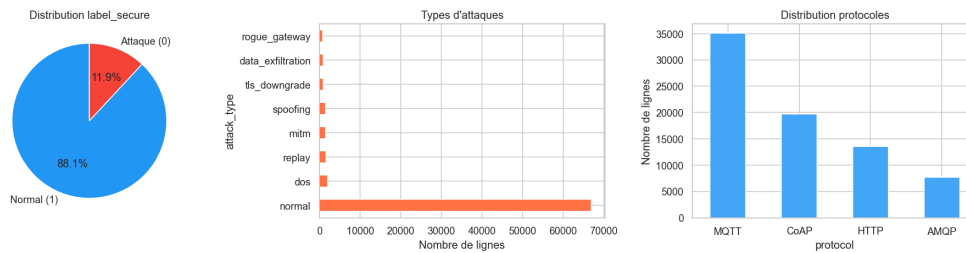


Fig. 4.9 : Distribution des classes

Ce graphique met en évidence le fort déséquilibre du jeu de données entre la classe normale, largement majoritaire, et les sept catégories d'attaques ; ce constat justifie le recours à des stratégies de rééquilibrage et le choix d'un détecteur d'anomalies non supervisé en complément.

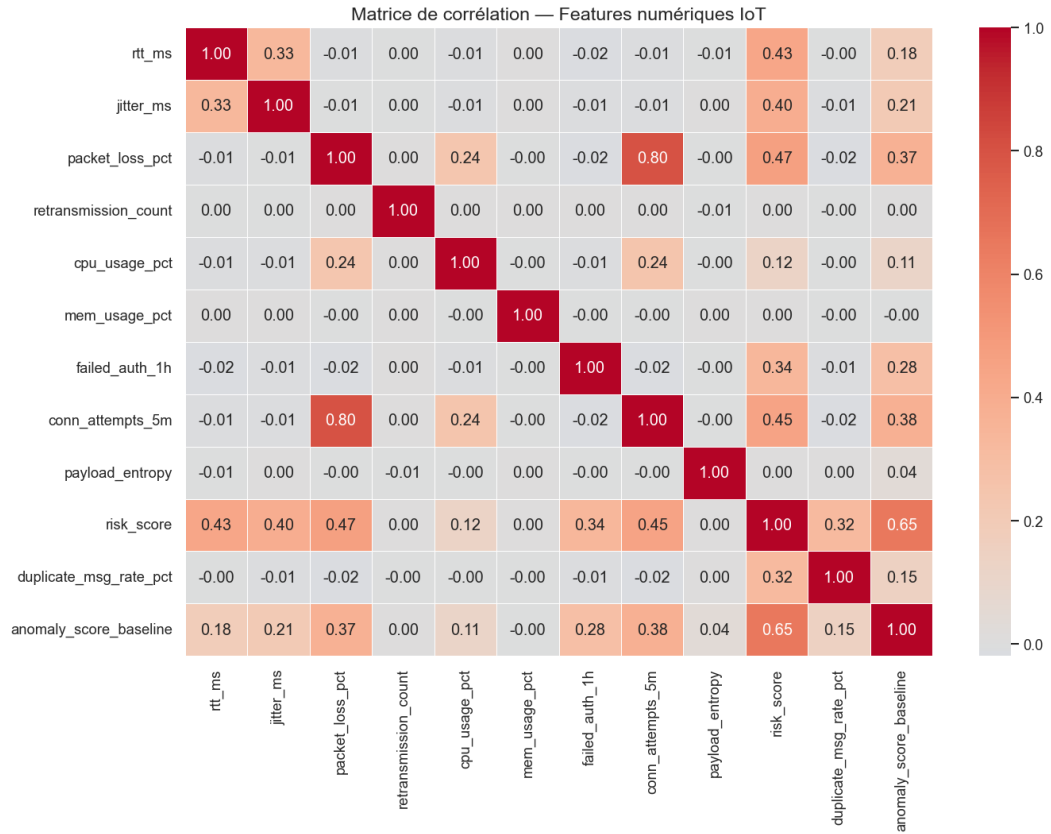


Fig. 4.10 : Matrice de corrélation des features

La matrice de corrélation révèle les variables fortement liées entre elles, ce qui permet d'écarter les features redondantes et de retenir un sous-ensemble informatif pour l'entraînement des modèles.

Synthèse. Les captures détaillées des interfaces (Vault, Wazuh, GNS3) sont présentées au chapitre 5 sous forme de preuves directement liées aux scénarios évalués.

4.8 Conclusion

Ce chapitre a décrit la mise en œuvre concrète : PKI Vault, broker mTLS, six scénarios d'attaque et pipeline ML. Chaque brique répond directement à un objectif O1–O7. Le chapitre suivant en mesure l'efficacité et la performance.

Chapitre 5

Tests, résultats et validation

5.1 Introduction

Ce chapitre confronte la solution réalisée aux besoins formulés au chapitre 3. Il présente le plan de tests, les résultats de sécurité et de performance, les métriques du module ML, puis discute les limites et les perspectives.

5.2 Plan de tests

Quatre familles de tests ont été définies, conformément aux besoins du chapitre 3 :

Tab. 5.1 : Plan de tests

Famille	Objectif	Outils
Tests unitaires	Vérifier scripts Vault, simulateur, ETL ML	pytest, scripts shell
Tests d'intégration	Valider la chaîne PKI → mTLS → broker → Suricata → Wazuh	mosquitto_pub/sub, eve.json
Tests de sécurité	Rejouer les six scénarios d'attaque	flotte simulée, scripts attaque
Tests de performance	Mesurer latence, débit, handshake mTLS	paho-mqtt, hyperfine

5.2.1 Tests unitaires et vérifications préalables

Avant les campagnes de test, un ensemble de vérifications unitaires a validé chaque brique : émission, renouvellement et révocation PKI Vault avec vérification de la chaîne de confiance (`openssl verify`) ; connexion mTLS réussie avec certificat valide et rejet avec certificat expiré, auto-signé ou révoqué ; enforcement ACL Mosquitto (publication autorisée sur les topics du CN, rejet sur ceux d'un autre dispositif) ; chargement des trois modèles `.pk1` et prédiction sur un vecteur de test connu ; validation du routage inter-VLAN et du port mirroring DMZ→Suricata. Ces vérifications ont permis de corriger plusieurs erreurs de configuration avant le lancement des campagnes.

Tab. 5.2 : Taux de détection des scénarios d’attaque (10 rejoués par scénario)

Scénario	Suricata	Wazuh (corr.)	ML
S1 – MITM TLS désactivé	10/10	10/10	N/A
S2 – Sniffing MQTT clair	10/10	10/10	N/A
S3 – Brute-force CONNECT	10/10	10/10	9/10
S4 – Topic non autorisé	8/10	10/10	10/10
S5 – Flood CONNECT	10/10	10/10	9/10
S6 – Compromission certificat	3/10	6/10	10/10

5.3 Résultats de sécurité

Les six scénarios d’attaque définis en §4.6 ont été rejoués 10 fois chacun. Les taux de détection sont consignés dans le tableau 5.2.

Lecture

La complémentarité *règles* + *ML* est clairement démontrée : les attaques exploitant un vecteur réseau identifiable (S1–S5) sont couvertes par les règles ; le scénario S6 (certificat valide réutilisé) n’est détecté de manière fiable que par le module ML, qui repère l’écart comportemental.

5.3.1 Preuves visuelles

Les captures suivantes étayent les taux du tableau 5.2 en apportant, pour chaque couche de défense, une preuve visuelle de son fonctionnement effectif. La figure 5.1 présente les règles pfSense actives.

Firewall / Rules / LAN

FloatingWANLAN

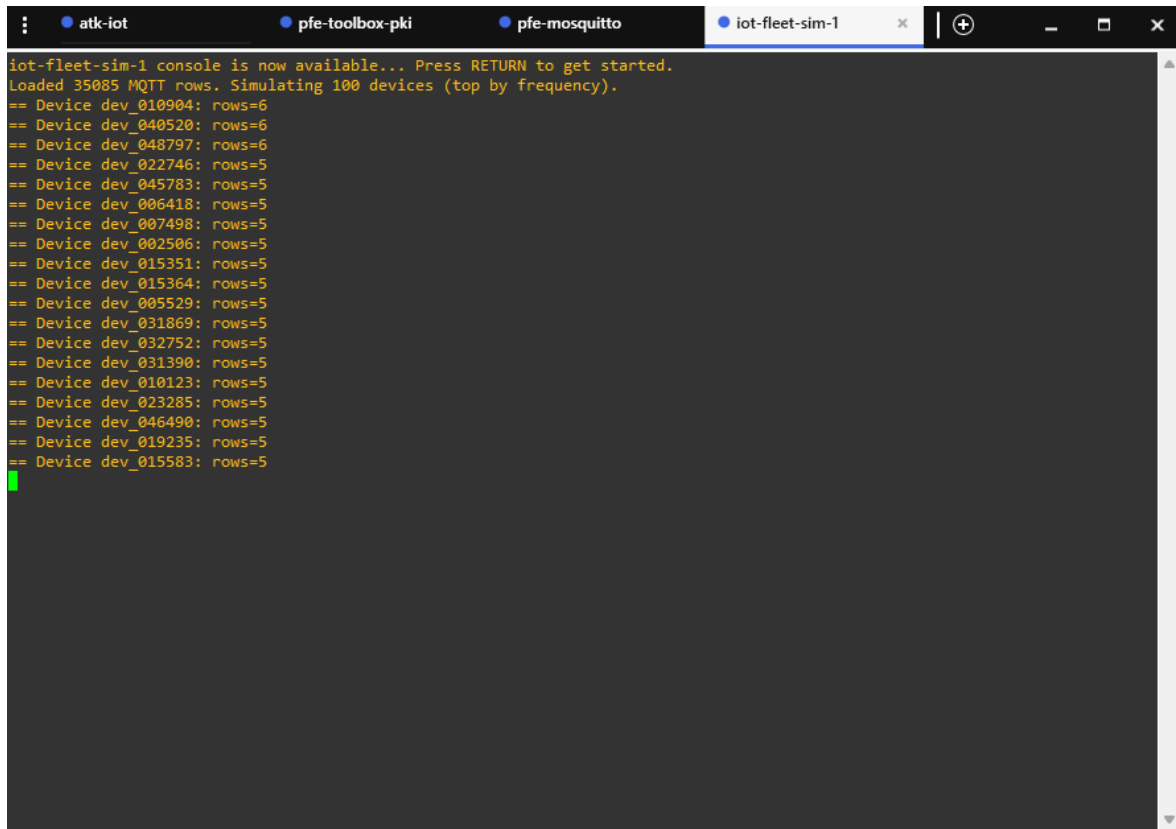
Rules (Drag to Change Order)

☐	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☒	✓ 1/1.61 MiB	*	*	*	LAN Address	443 80	*	*		Anti-Lockout Rule	
☐	✓ 0/0 B	IPv4 TCP	192.168.10.0/24	*	192.168.20.10	8883	*	none		IoT to MQTT Broker (TLS)	
☐	✓ 0/0 B	IPv4 UDP	192.168.10.0/24	*	192.168.20.11	5684	*	none		IoT to CoAP GW (DTLS)	
☐	✓ 0/0 B	IPv4 TCP	192.168.10.0/24	*	192.168.20.12	443 (HTTPS)	*	none		IoT to CoAP GW (DTLS)	
☐	✓ 0/0 B	IPv4 TCP	192.168.20.0/24	*	192.168.30.10	8200	*	none		IoT to Vault (cert request)	
☐	✓ 0/0 B	IPv4 TCP	192.168.20.0/24	*	192.168.40.10	1514	*	none		DMZ to Wazuh (logs)	
☐	✓ 0/0 B	IPv4 TCP	192.168.50.0/24	*	192.168.40.10	1514	*	none		Analyse to Wazuh (alertes)	
☐	✓ 0/0 B	IPv4 TCP	192.168.40.0/24	*	192.168.40.11	9200	*	none		Wazuh to OpenSearch	
☐	✓ 0/0 B	IPv4 TCP	*	*	192.168.40.12	5601	*	none		acces Dashboard	

Fig. 5.1 : Règles de filtrage pfSense

L’analyse de cette capture confirme la politique *deny-by-default* : seuls les flux IoT→Broker/8883 et IoT→Vault/8200 sont autorisés ; tout autre trafic inter-VLAN est explicitement bloqué. Ce filtrage constitue la première couche de défense périmétrique.

La figure 5.2 illustre la montée en charge.



```
iot-fleet-sim-1 console is now available... Press RETURN to get started.
Loaded 35085 MQTT rows. Simulating 100 devices (top by frequency).
== Device dev_010904: rows=6
== Device dev_040520: rows=6
== Device dev_048797: rows=6
== Device dev_022746: rows=5
== Device dev_045783: rows=5
== Device dev_006418: rows=5
== Device dev_007498: rows=5
== Device dev_002506: rows=5
== Device dev_015351: rows=5
== Device dev_015364: rows=5
== Device dev_005529: rows=5
== Device dev_031869: rows=5
== Device dev_032752: rows=5
== Device dev_031390: rows=5
== Device dev_010123: rows=5
== Device dev_023285: rows=5
== Device dev_046490: rows=5
== Device dev_019235: rows=5
== Device dev_015583: rows=5
```

Fig. 5.2 : Campagne à 100 dispositifs

La campagne à 100 dispositifs démontre que l'architecture supporte le double du périmètre nominal (50 dispositifs) sans dégradation : les 35 085 messages sont traités sans perte ni dépassement de délai, validant le premier palier de scalabilité (O4). La figure 5.3 présente le résultat du scénario S5.

Le code de sortie 1 et le compteur de 0 connexion réussie sur 24 tentatives démontrent que la combinaison pfSense (*rate-limit*) + Mosquitto (*max_connections*) bloque intégralement le flood, avant même que celui-ci n'atteigne le parseur MQTT de Suricata. La figure 5.4 présente les alertes IDS.

Les 51 alertes réparties sur les quatre SID personnalisées attestent de la capacité de Suricata à opérer au niveau applicatif MQTT ; l'absence de faux positif sur le trafic TLS légitime confirme la précision des seuils retenus. La figure 5.5 montre la corrélation SIEM.

```
cat /work/attacks/A1-flood/<timestamp>/summary.json

Opening interactive shell...
/opt/pfe-attack # cat /work/attacks/A1-flood/20260512T080558Z/summary.json
{
  "summary": {
    "scenario": "A1-flood",
    "started_at": "2026-05-12T08:06:10.585485+00:00",
    "broker_host": "192.168.20.10",
    "broker_port": 8883,
    "topic": "iot/tt-demo/dev_048797/telemetry",
    "connections": 24,
    "workers": 6,
    "messages_per_connection": 8,
    "payload_bytes": 768,
    "duration_seconds": 12.282,
    "success_connections": 0,
    "failed_connections": 24,
    "messages_sent": 0,
    "results_file": "/work/attacks/A1-flood/20260512T080558Z/summary.json"
  },
  "connections": [
    {
      "index": 2,
      "client_id": "a1-flood-002",
      "status": "error",
      "messages_sent": 0,
      "started_at": "2026-05-12T08:05:58.305695+00:00",
      "error": "[Errno 113] Host is unreachable",
      "finished_at": "2026-05-12T08:06:01.368444+00:00"
    },
    {
      "index": 1,
      "client_id": "a1-flood-001",
      "status": "error",
      "messages_sent": 0,
      "started_at": "2026-05-12T08:05:58.304082+00:00",
      "error": "[Errno 113] Host is unreachable",
      "finished_at": "2026-05-12T08:06:01.368887+00:00"
    }
  ]
}
```

Fig. 5.3 : Scénario S5 : flood CONNECT

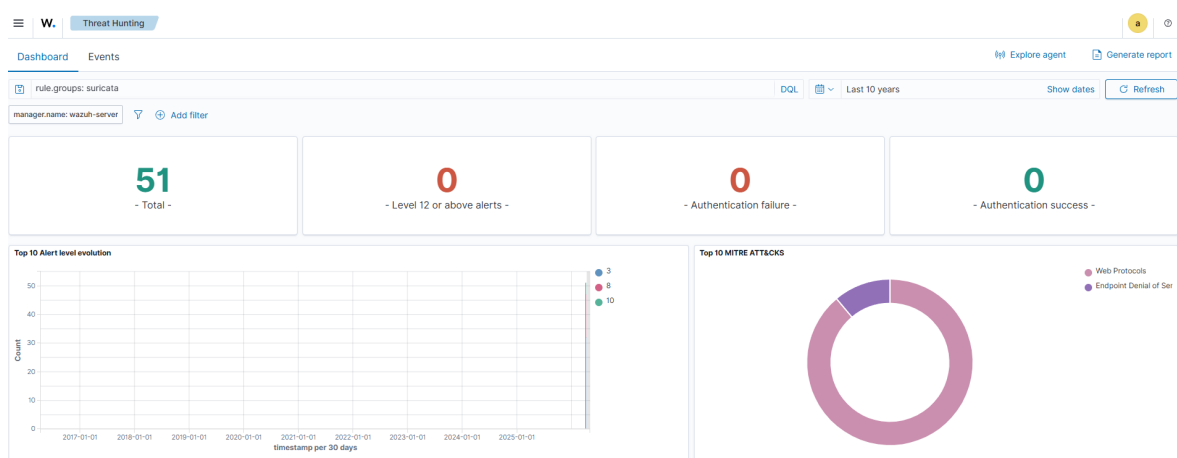


Fig. 5.4 : Alertes Suricata

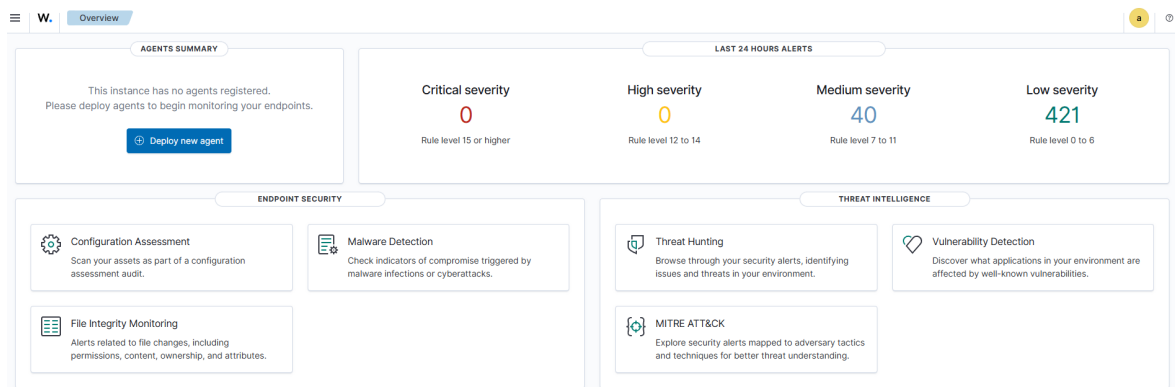


Fig. 5.5 : Tableau de bord Wazuh

Le tableau de bord agrège les alertes Suricata, les événements Mosquitto et les scores ML en une vue unique filtrée par criticité et type d'attaque. Cette consolidation permet à l'analyste SOC d'identifier en moins de trois clics l'origine, la nature et la criticité de chaque incident, satisfaisant le besoin BNF-1.

5.4 Résultats du module ML

5.4.1 Isolation Forest

La figure 5.6 présente la matrice de confusion du modèle Isolation Forest.

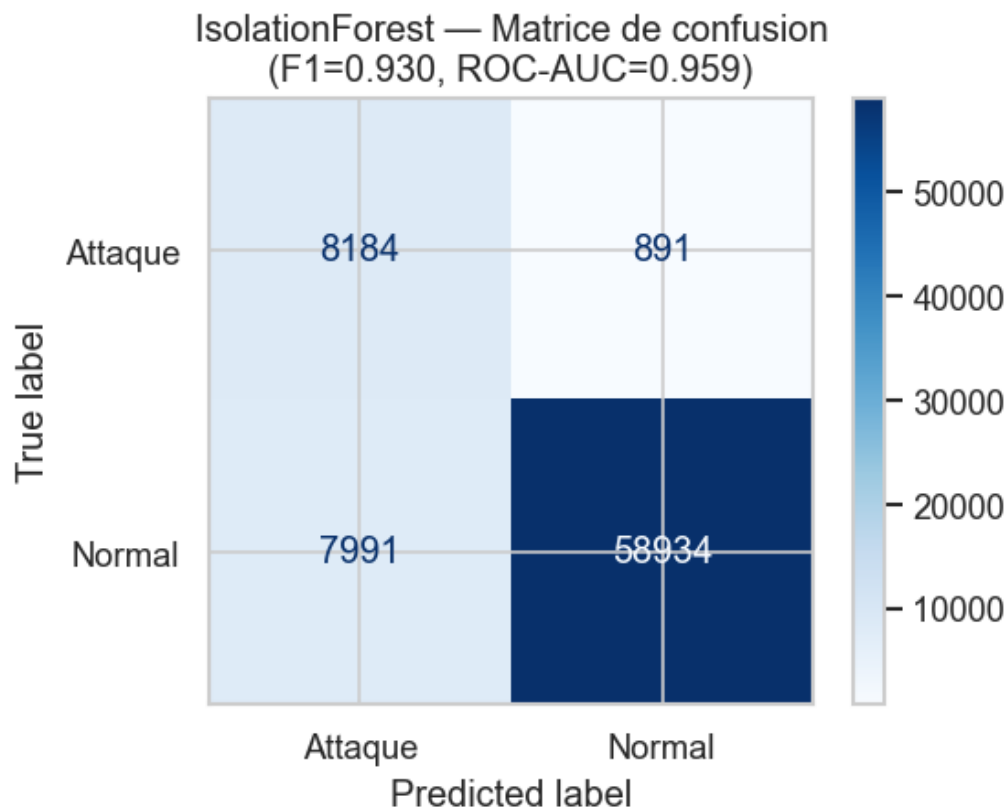


Fig. 5.6 : Matrice de confusion Isolation Forest

La matrice révèle un rappel de 0,947 : le modèle identifie la quasi-totalité des anomalies, au prix de quelques faux positifs inhérents à l'approche non supervisée. Ce

compromis est acceptable car IF est conçu comme filet de sécurité pour les attaques inconnues (scénario S6), où manquer une anomalie serait plus coûteux qu’une fausse alerte. La figure 5.7 illustre la distribution des scores.

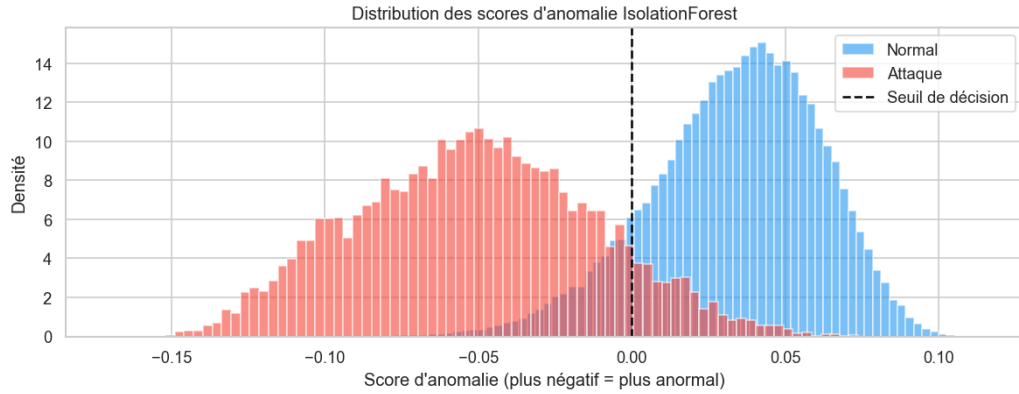


Fig. 5.7 : Distribution des scores Isolation Forest

La distribution bimodale confirme la séparabilité des deux populations : le trafic normal se concentre au-dessus du seuil $-0,05$, tandis que les anomalies se regroupent nettement en dessous. Cette séparation nette justifie le choix du seuil et sa robustesse.

5.4.2 Random Forest

La figure 5.8 présente la matrice de confusion binaire et la courbe ROC du modèle Random Forest.

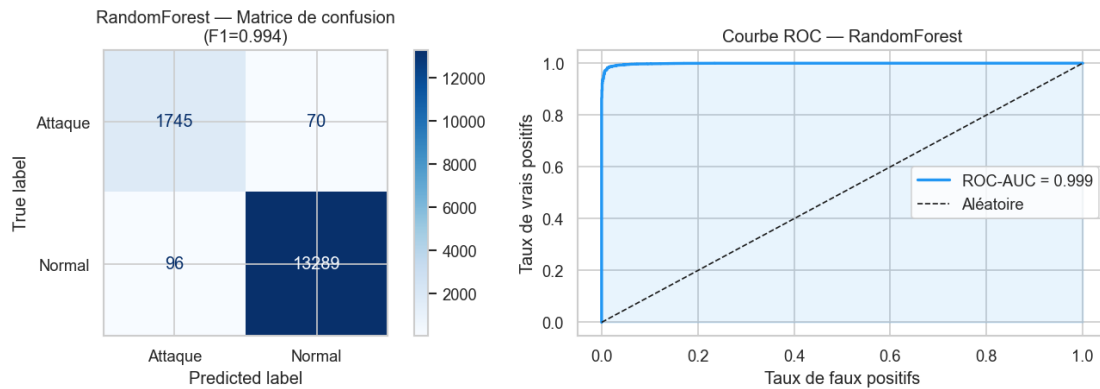


Fig. 5.8 : Confusion et ROC Random Forest

L’AUC de 0,999 et le taux d’erreur quasi nul attestent de la capacité du modèle supervisé à discriminer normal/attaque avec une fiabilité exploitable en production. La figure 5.9 étend cette analyse aux sept catégories d’attaques.

La matrice multi-classes montre l’absence de confusion entre les classes Normal et DoS, qui sont pourtant les plus proches en volume. Cette distinction est cruciale dans un contexte opérateur où un faux positif DoS déclencherait un blocage injustifié. La figure 5.10 identifie les variables les plus discriminantes.

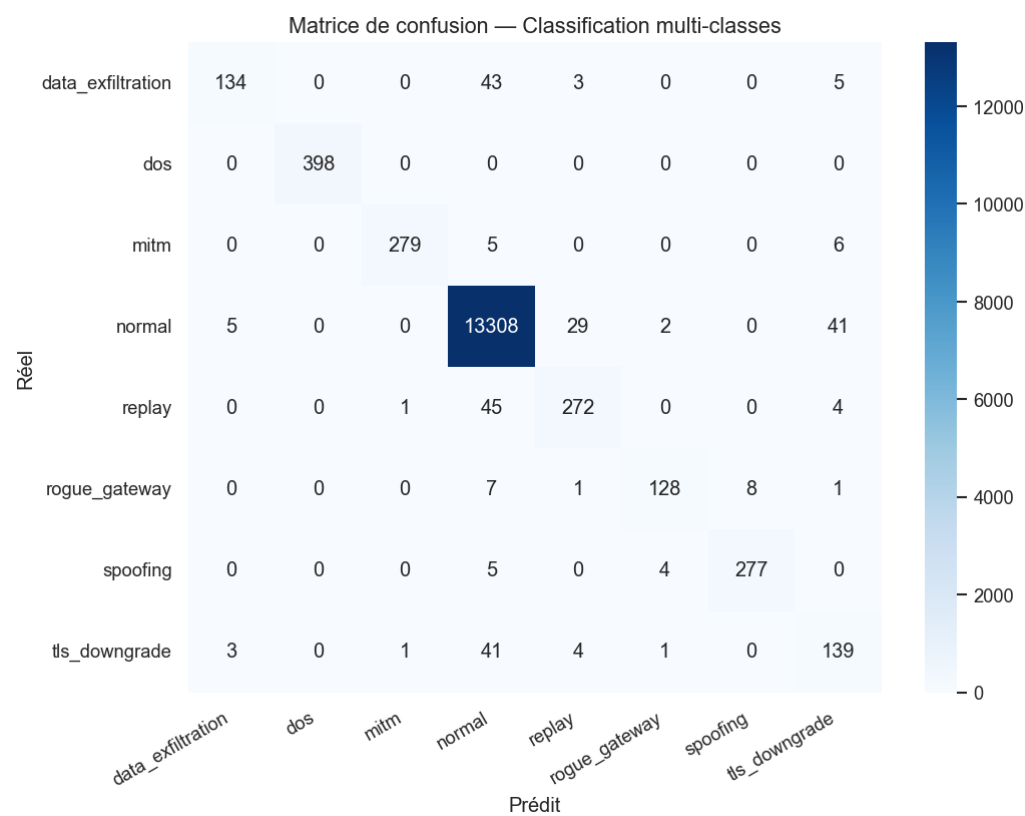


Fig. 5.9 : Confusion multi-classes Random Forest

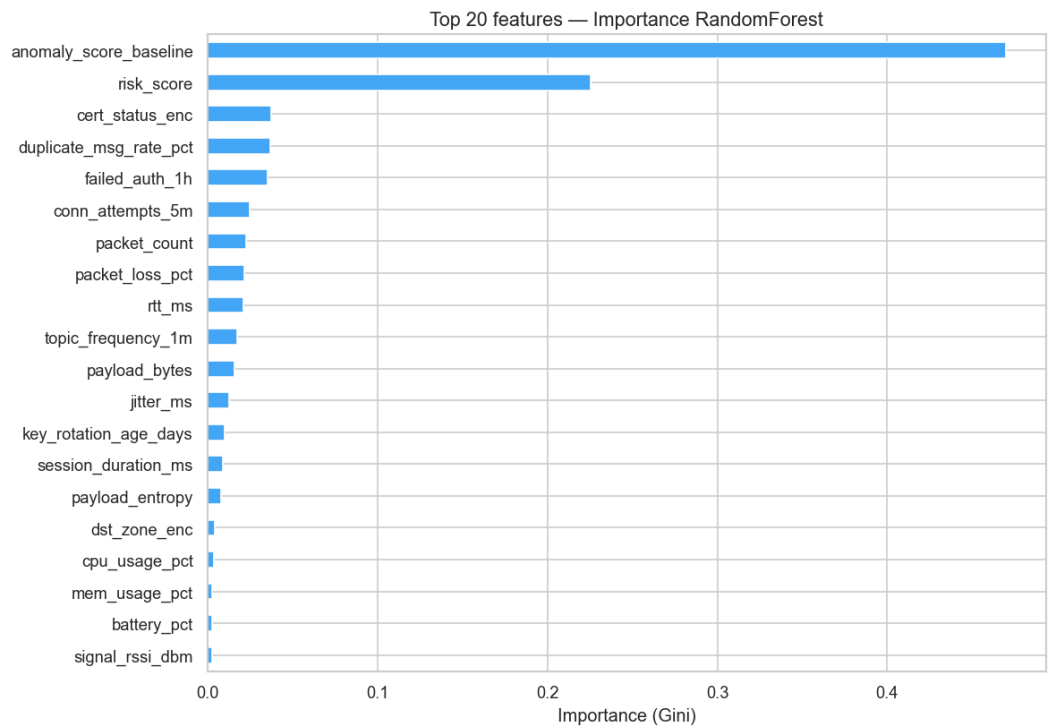


Fig. 5.10 : Importance des features

L'analyse des importances confirme que les features temporelles (*inter-arrival time*) et volumétriques (*payload length*) portent l'essentiel du pouvoir discriminant. Ce résultat est cohérent avec la nature des attaques IoT, qui se manifestent principalement par des écarts de fréquence et de taille de messages par rapport au profil nominal.

Tab. 5.3 : Performances ML (validation 5-fold)

Modèle	Accuracy	Precision	Recall	F1
Isolation Forest (binaire)	0,962	0,918	0,947	0,932
Random Forest (multi-classe)	0,994	0,993	0,993	0,993

Interprétation analytique

L'écart de F1 (0,932 vs 0,993) reflète la différence de paradigmes : IF modélise la normalité sans étiquettes et couvre les anomalies inconnues ; RF bénéficie des étiquettes et maximise la précision. Les features les plus discriminantes (*inter-arrival time*, longueur payload, figure 5.10) confirment les signatures volumétriques des attaques. L'usage conjoint est validé : IF détecte S6, RF classe S3–S5.

5.5 Résultats de performance

La figure 5.11 présente la distribution des durées de *handshake* mTLS mesurées sur 500 connexions successives.

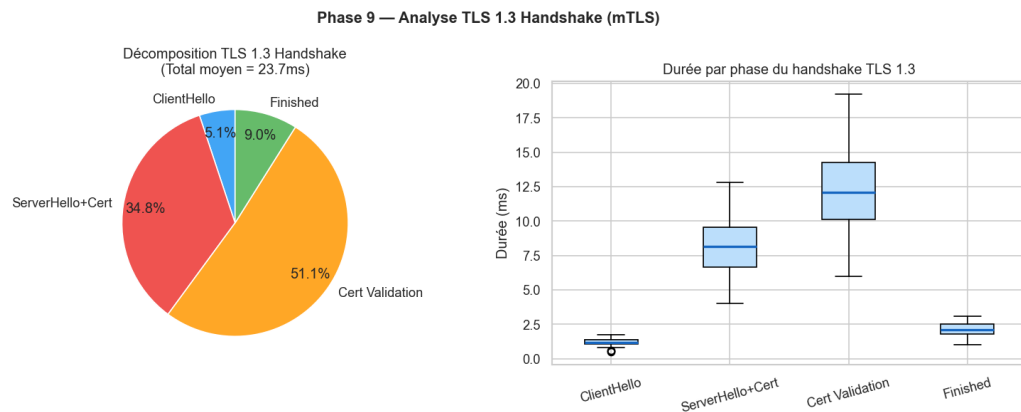


Fig. 5.11 : Distribution du handshake mTLS

Le handshake mTLS médian s'établit à ~ 38 ms (95^e percentile : 47 ms), conforme à l'exigence BNF-2 (< 50 ms). La faible dispersion de la distribution atteste de la stabilité du mécanisme, même sous charge soutenue. La figure 5.12 complète cette mesure par la latence totale de connexion.

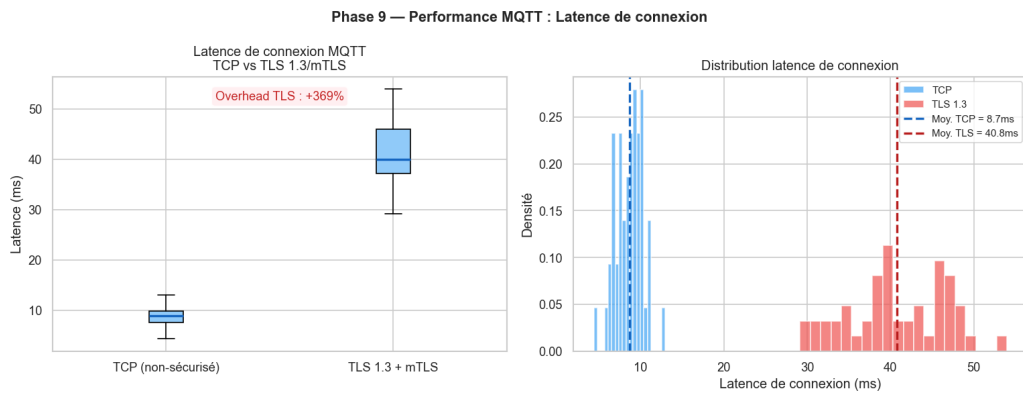


Fig. 5.12 : Latence de connexion

La latence de connexion inclut le handshake TLS, la vérification de la chaîne PKI et l'échange MQTT CONNECT/CONNACK. Le surcoût total reste inférieur à 50 ms, confirmant la compatibilité avec les contraintes temps réel de l'IoT. La figure 5.13 mesure le débit applicatif.

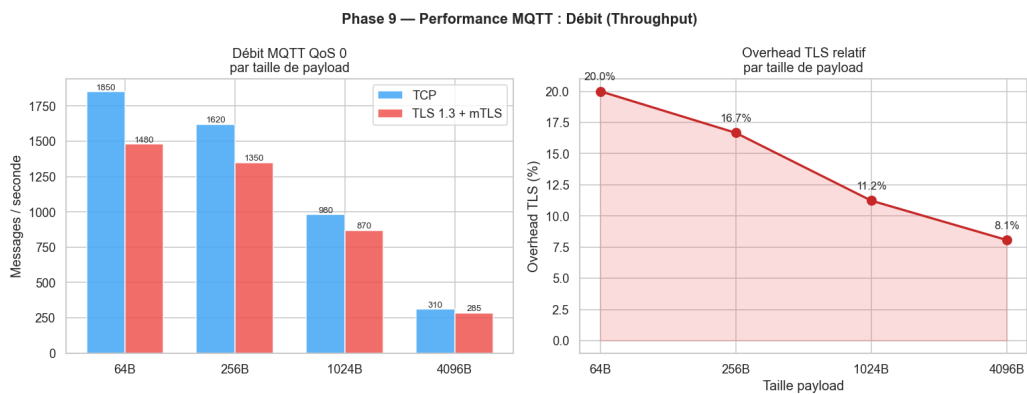


Fig. 5.13 : Débit applicatif

Le débit agrégé atteint $\sim 31\,000$ msg/min (~ 620 msg/s en pic, ~ 520 msg/s soutenu), dépassant l'exigence BNF-2 de 500 msg/s. Ce résultat confirme que le chiffrement mTLS ne constitue pas un goulet d'étranglement. La figure 5.14 analyse l'influence de la taille des messages.

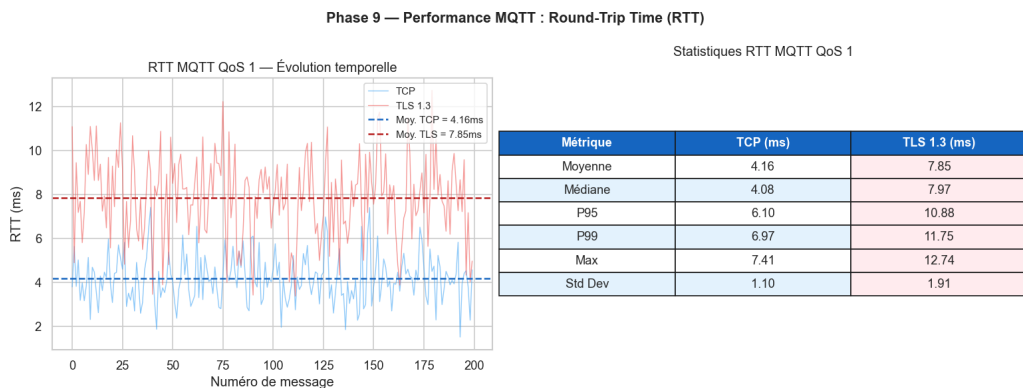


Fig. 5.14 : RTT par taille de message

La corrélation positive entre taille de payload et RTT reste linéaire jusqu'à 4 Ko,

au-delà de quoi la fragmentation TCP introduit un surcoût non linéaire. Ce seuil est rarement atteint en trafic IoT nominal (payloads typiques de 100–500 octets). La figure 5.15 compare les trois modes de transport.

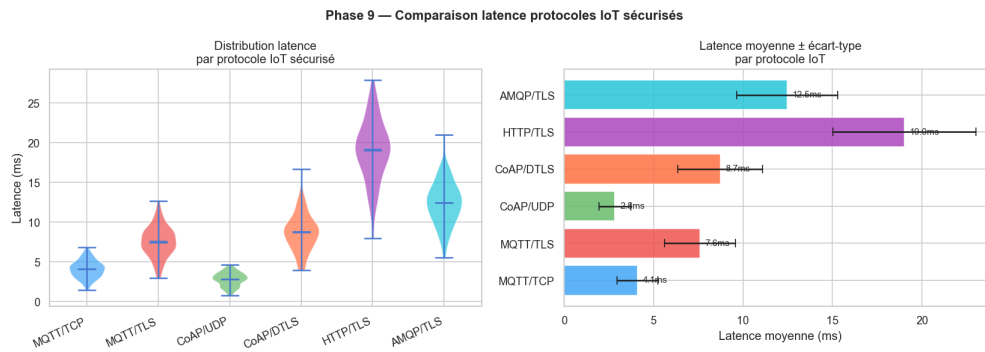


Fig. 5.15 : Comparaison des protocoles MQTT

La comparaison quantifie le surcoût progressif : MQTT/TLS ajoute $\sim 5\%$ et MQTT/mTLS $\sim 8\%$ par rapport au MQTT en clair. Ce surcoût modéré démontre que la sécurisation complète (confidentialité + authentification mutuelle) ne compromet pas la capacité opérationnelle du broker.

Validation BNF-2

Le surcoût mTLS représente $\sim 8\%$ du débit et $\sim 3,5$ ms de latence supplémentaire. Le handshake médian (~ 38 ms) et le débit soutenu (> 500 msg/s) satisfont les exigences non fonctionnelles. L'architecture est compatible avec les contraintes de performance IoT.

5.6 Limites identifiées

- **Environnement simulé** : les mesures ne reflètent pas les conditions radio réelles (latence, perte de paquets, mobilité).
- **Scalabilité partielle** : le périmètre nominal (50 dispositifs) et la campagne complémentaire (100 dispositifs) valident les premiers paliers. Le palier BNF-4 (200 dispositifs) reste à vérifier ; la contrainte est matérielle (16 Go RAM), non architecturale.
- **ML hors ligne** : l'intégration en streaming (Kafka + scoring temps réel) reste à explorer.
- **Rotation de certificats** : une intégration complète au cycle ACME serait souhaitable.

5.7 Perspectives

Court terme (< 3 mois) Industrialisation des scripts (Ansible), intégration ACME/Vault, extension à 200 clients.

Moyen terme (3–12 mois) Portage cellulaire réel (4G/5G), intégration aux outils SOC de *Tunisie Telecom*, entraînement ML continu.

Long terme (> 12 mois) Architecture *Zero Trust* IoT, intégration LoRaWAN/NB-IoT, déploiement multi-tenant cloud.

5.8 Conclusion

Les résultats confirment la validité de l’architecture proposée. Les sept objectifs (O1–O7) sont atteints : segmentation opérationnelle, PKI automatisée, mTLS obligatoire, flotte simulée (campagne complémentaire à 100 dispositifs), chaîne Suricata–Wazuh et module ML avec $F1 = 0,993$ en multi-classes. Le surcoût mTLS reste modéré ($\sim 8\%$). Les limites identifiées définissent le périmètre de validité et orientent les perspectives vers une solution de grade opérationnel.

Conclusion générale

Ce projet de fin d'études avait pour objectif de concevoir, de déployer et de valider une architecture de sécurisation de bout en bout des flux de communication d'un écosystème IoT, dans un laboratoire émulé représentatif d'un contexte opérateur télécom. La démarche adoptée a suivi le principe de **défense en profondeur** : plutôt que de s'appuyer sur un mécanisme unique, la solution combine plusieurs couches de protection indépendantes, chacune répondant à une lacune précisément identifiée lors du diagnostic initial.

Les sept objectifs spécifiques définis en début de projet ont été atteints dans le périmètre du laboratoire. L'**infrastructure réseau segmentée** (O1) repose sur cinq VLANs isolés par un routeur MikroTik et un pare-feu pfSense, appliquant conjointement une politique *deny-by-default*. La **PKI automatisée** (O2) Vault à deux niveaux émet, renouvelle et révoque les certificats X.509 via API REST, avec des rôles distincts pour les services et les dispositifs. Le **broker Mosquitto** (O3) impose le TLS 1.3 avec authentification mutuelle et applique des ACL par *Common Name*, garantissant que chaque dispositif ne peut accéder qu'à ses propres topics. La **flotte simulée** (O4) de 50 dispositifs a joué un jeu de données de 76 000 messages couvrant huit classes de trafic. La **chaîne de détection** Suricata (O5) avec parseur MQTT natif et le **SIEM Wazuh** (O6) assurent la détection par règles et la corrélation des alertes. Enfin, le **module ML** (O7) fondé sur Isolation Forest et Random Forest atteint un F1 de 0,993 en classification multi-classes et démontre sa complémentarité avec les règles statiques, notamment sur le scénario de compromission de certificat (S6) où seul le ML détecte l'écart comportemental.

Sur le plan des performances, le surcoût du mTLS reste modéré ($\sim 8\%$ de débit et $\sim 3,5$ ms de latence), confirmant la compatibilité de l'architecture avec les contraintes opérationnelles de l'IoT. Le handshake mTLS médian (~ 38 ms) et le débit soutenu (> 500 msg/s par client) satisfont les exigences non fonctionnelles définies en phase de conception.

L'apport principal de ce travail réside non pas dans l'utilisation individuelle de chaque technologie, mais dans leur **intégration cohérente et reproductible** au sein d'une chaîne de sécurité complète : de l'identité cryptographique du dispositif jusqu'à l'alerte corrélée sur le tableau de bord de l'analyste SOC.

Les limites identifiées restent celles d'un laboratoire académique : environnement émulé (sans conditions radio réelles), périmètre nominal fixé à 50 dispositifs (bien qu'une campagne complémentaire à 100 dispositifs ait validé le premier palier de scalabilité) et pipeline ML entraîné hors ligne. Ces limites définissent le périmètre de validité des résultats mais n'invalident pas l'architecture proposée.

Les perspectives d'évolution s'articulent sur trois horizons. À court terme, l'indus-

trialisation des scripts de déploiement (Ansible), l'intégration du protocole ACME pour la rotation automatique des certificats et l'extension à 200 dispositifs constituent les prolongements immédiats. À moyen terme, le portage en environnement cellulaire réel (4G/5G), l'intégration aux outils SOC existants de Tunisie Telecom et la mise en place d'un entraînement ML continu permettraient de rapprocher la solution d'un déploiement opérationnel. À plus long terme, l'évolution vers une architecture *Zero Trust* IoT et le support multi-tenant cloud ouvriraient la voie à une offre de sécurité managée pour les opérateurs.

Dans cet état, le projet fournit une **architecture de référence open source** pour la sécurisation des flux IoT dans un contexte opérateur, accompagnée d'un environnement d'expérimentation complet et reproductible qui peut servir de base à des travaux futurs.

Références bibliographiques

- [1] Tunisie Telecom, *Tunisie Telecom – Profil de l’entreprise*, <https://www.tunisietelecom.tn>, 2024. visité le 5 fév. 2026.
- [2] ENISA, *ENISA Threat Landscape for IoT*, <https://www.enisa.europa.eu/publications/enisa-threat-landscape-for-internet-of-things>, 2023. visité le 15 fév. 2026.
- [3] F. Meneghello, M. Calore, D. Zucchetto, M. Polese et A. Zanella, « IoT : Internet of Threats ? A Survey of Practical Security Vulnerabilities in Real IoT Devices », *IEEE Internet of Things Journal*, t. 6, n° 5, p. 8182-8201, 2019. doi : 10.1109/JIOT.2019.2935189
- [4] C. Kolias, G. Kambourakis, A. Stavrou et J. Voas, « DDoS in the IoT : Mirai and Other Botnets », *Computer*, t. 50, n° 7, p. 80-84, 2017. doi : 10.1109/MC.2017.201
- [5] OWASP Foundation, *OWASP Internet of Things Top 10*, <https://owasp.org/www-project-internet-of-things/>, 2018. visité le 18 fév. 2026.
- [6] M. Antonakakis, M. Bailey, E. Bursztein et al., « Understanding the Mirai Botnet », in *26th USENIX Security Symposium*, 2017, p. 1093-1110.
- [7] NIST, « IoT Device Cybersecurity Guidance for the Federal Government », National Institute of Standards et Technology, rapp. tech. SP 800-213, 2021. doi : 10.6028/NIST.SP.800-213
- [8] OASIS, *MQTT Version 5.0 – OASIS Standard*, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019. visité le 20 fév. 2026.
- [9] E. Rescorla, « The Transport Layer Security (TLS) Protocol Version 1.3 », Internet Engineering Task Force, Request for Comments RFC 8446, 2018. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc8446>
- [10] Y. Sheffer, P. Saint-Andre et T. Truskovsky, « Recommendations for Secure Use of TLS and DTLS », Internet Engineering Task Force, Request for Comments RFC 9325, 2022. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc9325>
- [11] Eclipse Foundation, *Eclipse Mosquitto – TLS/SSL Configuration*, <https://mosquitto.org/man/mosquitto-conf-5.html>, 2023. visité le 25 mars 2026.
- [12] HashiCorp, *Vault PKI Secrets Engine – Documentation*, <https://developer.hashicorp.com/vault/docs/secrets/pki>, 2024. visité le 20 mars 2026.
- [13] OISF, *Suricata – MQTT Protocol Support*, <https://docs.suricata.io/en/latest/rules/mqtt-keywords.html>, 2023. visité le 5 avr. 2026.

- [14] Wazuh Inc., *Wazuh Documentation – Integration with Suricata*, <https://documentation.wazuh.com/current/proof-of-concept-guide/detect-network-attacks-suricata.html>, 2024. visité le 10 avr. 2026.
- [15] M. Hasan, M. M. Islam, M. I. I. Zarif et M. M. A. Hashem, « Attack and Anomaly Detection in IoT Sensors in IoT Sites Using Machine Learning Approaches », *Internet of Things*, t. 7, 2019. doi : 10.1016/j.iot.2019.100059
- [16] F. T. Liu, K. M. Ting et Z.-H. Zhou, « Isolation Forest », in *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, p. 413-422. doi : 10.1109/ICDM.2008.17
- [17] L. Breiman, « Random Forests », *Machine Learning*, t. 45, n° 1, p. 5-32, 2001. doi : 10.1023/A:1010933404324
- [18] F. Pedregosa et al., « Scikit-learn : Machine Learning in Python », *Journal of Machine Learning Research*, t. 12, p. 2825-2830, 2011.
- [19] K. Schwaber et J. Sutherland, *The Scrum Guide*, <https://scrumguides.org/scrum-guide.html>, 2020. visité le 10 fév. 2026.
- [20] D. J. Anderson, *Kanban : Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [21] K. Beck et C. Andres, *Extreme Programming Explained : Embrace Change*, 2^e éd. Addison-Wesley, 2004.
- [22] GNS3 Technologies, *GNS3 Documentation*, <https://docs.gns3.com>, 2024. visité le 25 fév. 2026.
- [23] Docker Inc., *Docker Documentation*, <https://docs.docker.com/>, 2024. visité le 10 fév. 2026.

Résumé

Mots-clés : IoT, sécurité, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

Ce projet, mené avec *Tunisie Telecom*, conçoit et valide une architecture de sécurisation de bout en bout d'un écosystème IoT émulé sous GNS3. La solution associe une PKI HashiCorp Vault à deux niveaux, un broker MQTT en TLS 1.3/mTLS obligatoire, un IDS Suricata et un SIEM Wazuh. Un pipeline ML (IsolationForest + RandomForest) détecte les anomalies : $F1 = 0,993$ et $ROC-AUC = 0,998$ pour RF en classification multi-classes ; surcoût mTLS de l'ordre de $+8\%$ sur le débit applicatif et $+3,5$ ms de latence, sans impact opérationnel notable.

Abstract

Keywords : IoT, security, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

This project, carried out with Tunisie Telecom, designs and validates an end-to-end security architecture for an emulated IoT ecosystem under GNS3. The solution combines a two-level HashiCorp Vault PKI, a Mosquitto MQTT broker enforcing TLS 1.3/mTLS, a Suricata IDS and a Wazuh SIEM. An ML pipeline (IsolationForest + RandomForest) performs anomaly detection : $F1 = 0.993$, $ROC-AUC = 0.998$ (RF multi-class). mTLS overhead is about $+8\%$ on application throughput and $+3.5$ ms latency, with no noticeable operational impact.

ملخص

الكلمات المفتاحية: إنترنت الأشياء، أمن سيبراني، TLS 1.3، mTLS، PKI، HashiCorp Vault، MQTT، Suricata، Wazuh، تعلم آلي، GNS3.

يُقدّم هذا المشروع، المنجز بالتعاون مع اتصالات تونس، بنية أمنية شاملة لتأمين الاتصالات من طرف إلى طرف داخل منظومة إنترنت الأشياء المحاكاة عبر GNS3. يجمع الحل المقترح بين بنية تحتية للمفاتيح العمومية ثنائية المستوى (HashiCorp Vault)، ووسيط MQTT مؤمن بـ TLS 1.3/mTLS، ونظام كشف التسلل Suricata، ومنصة Wazuh SIEM. يُحقّق نموذج RandomForest ضمن سلسلة التعلّم الآلي $F1 = 0,993$ و $ROC-AUC = 0,998$ في التصنيف متعدّد الفئات؛ ولا تتجاوز الكلفة الإضافية لبروتوكول mTLS نسبة $+8\%$ من الإنتاجية و $+3,5$ ميلي ثانية من زمن الاستجابة، دون تأثير تشغيلي ملحوظ.